

Universidad Politécnica de Madrid

Escuela Técnica Superior de Ingenieros Informáticos



Máster en Software y Sistemas

Trabajo Fin de Máster

Empirical Study of Third-Party Library Detection in iOS and Android Applications

Autor: Pablo Marchand Ramirez
Tutor(a): Alessandra Gorla

Madrid, June 2024

Este Trabajo Fin de Máster se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Trabajo Fin de Máster
Máster en Software y Sistemas

Título: Empirical Study of Third-Party Library Detection in iOS and Android Applications

June 2024

Autor: Pablo Marchand Ramirez
Tutor: Alessandra Gorla
Imdea Software
Escuela Técnica Superior de Ingenieros Informáticos
Universidad Politécnica de Madrid

Resumen

La integración de bibliotecas externas en el desarrollo de aplicaciones móviles se ha convertido en una práctica estándar, proporcionando a los desarrolladores funcionalidades preconstruidas que reducen significativamente el tiempo y el esfuerzo de desarrollo. Sin embargo, la dependencia en bibliotecas externas introduce diversos desafíos, incluyendo vulnerabilidades de seguridad y privacidad. En consecuencia, existe la necesidad de herramientas efectivas para detectarlas dentro de las aplicaciones móviles. Esta tesis presenta una guía completa de herramientas de detección de bibliotecas externas para las plataformas de Android e iOS, con el objetivo de asistir a los desarrolladores en la selección de la herramienta más adecuada para sus requisitos específicos.

Nuestro objetivo principal es evaluar y analizar las herramientas de detección de bibliotecas externas existentes mediante enfoques tanto teóricos como empíricos. El análisis teórico examina la arquitectura, las características y las metodologías de varias herramientas, ofreciendo una comprensión profunda de sus capacidades y limitaciones. Las evaluaciones empíricas involucran experimentos que prueban la efectividad, eficiencia, precisión en la detección de versiones, resistencia a la ofuscación y usabilidad de las herramientas.

Además, tomando las herramientas con mejores resultados para Android e iOS, se realizó un análisis en un conjunto de las mismas aplicaciones para ambas plataformas con la finalidad de obtener más información sobre las diferencias entre Android e iOS. Entre las diferencias más notables destaca el número de bibliotecas externas que en algun momento tuvieron vulnerabilidades por aplicación, donde Android triplica las de iOS, mostrando un caso de uso significativo para las herramientas de detección de bibliotecas externas.

Dado el complejo análisis de herramientas y este caso de uso, se espera que los usuarios puedan elegir fácilmente qué herramienta utilizar, así como aplicarla en cualquier situación donde sea necesario. Con esta guía y la evaluación detallada, los desarrolladores podrán tomar decisiones más informadas y estratégicas en la selección de herramientas de detección de bibliotecas externas.

Abstract

The integration of third-party libraries (TPLs) in mobile app development has become a standard practice, providing developers with pre-built functionalities that significantly reduce development time and effort. However, reliance on TPLs introduces various challenges, including security vulnerabilities, compliance issues, and difficulties in managing and updating the libraries. Consequently, there is a need for effective tools to detect TPLs within mobile applications. This thesis presents a comprehensive guide to TPL detection tools for both Android and iOS platforms, aiming to assist developers in selecting the most suitable tool for their specific requirements.

Our primary objective is to evaluate and analyze existing TPL detection tools through both theoretical and empirical approaches. The theoretical analysis examines the architecture, features, and methodologies of various tools, offering a deep understanding of their capabilities and limitations. Empirical evaluations involve experiments testing the effectiveness, efficiency, accuracy in version detection, resilience to obfuscation, and usability of the tools.

Additionally, by taking the tools that performed best for Android and iOS, an analysis was conducted on a set of the same applications for both platforms to gain further insights into the differences between Android and iOS. Among the most notable differences is the number of TPLs which had vulnerabilities per application, which in Android triples that of iOS, demonstrating a significant use case for TPL detection tools.

Given the complex analysis of tools and this use case, it is expected that users will be able to easily choose which tool to use, as well as apply it in any situation where it is necessary. With this guide and detailed evaluation, developers will be able to make more informed decisions in selecting TPL detection tools.

Contents

1 Introduction **1**
1.1 Context . . . 1
1.2 Motivation . . . 1
1.3 Objectives . . . 2

2 Background on Mobile App Development **5**
2.1 iOS . . . 5
2.1.1 Development Environment and Tools . . . 5
2.1.2 Structure of Binaries . . . 6
2.1.3 Security Features . . . 7
2.2 Android . . . 7
2.2.1 Development Environment and Tools . . . 8
2.2.2 Structure of Binaries . . . 8
2.2.3 Security Features . . . 9

3 State of the Art on Third-party Library Detection Tools **11**
3.1 Overview of Libraries in Mobile App Development . . . 11
3.1.1 Overview of Libraries in Mobile App Development . . . 11
3.1.2 Third-Party Libraries in iOS . . . 11
3.1.3 Third-Party Libraries in Android . . . 13
3.2 Third-Party Library Detection Techniques . . . 14
3.3 iOS Detection Tools . . . 15
3.3.1 CRiOS . . . 15
3.3.2 iLibScope . . . 16
3.3.3 LibKit . . . 18
3.4 Android Detection Tools . . . 20
3.4.1 AdDetect . . . 20
3.4.2 LibRadar . . . 22
3.4.3 LibScout . . . 23
3.4.4 ORLIS . . . 25
3.4.5 LibID . . . 26
3.4.6 LIBLOOM . . . 30
3.4.7 LibScan . . . 32

4 Empirical comparison between tools **35**
4.1 Research experiments . . . 35

4.2 Android tools 36
4.2.1 Database building 36
4.2.2 Effectiveness Evaluation 37
4.2.3 Efficiency Evaluation 39
4.2.4 Capability of Version Identification 40
4.2.5 Obfuscation-Resilience Capability 41
4.3 iOS tools 44
4.3.1 Experimental Setup 44
4.3.2 Ground Truth 44
4.3.3 Effectiveness Evaluation 44
4.3.4 Efficiency Evaluation 45
4.3.5 Capability of Version Identification 46
4.4 User Experience Evaluation 47

5 Comparison of TPL Usage Between Android and iOS **51**
5.1 Number of TPL per app 53
5.2 Types of TPL Utilized 54
5.3 Vulnerability per Application Comparison 57

6 Conclusion **59**
6.1 Objectives Review 59
6.2 Personal Reflections 60

Bibliography **63**

Chapter 1

Introduction

1.1 Context

In today's digital era, mobile applications have become indispensable, significantly transforming communication, productivity, and entertainment. The dominance of two primary operating systems, Android and iOS, has led to the development of a vast ecosystem of mobile applications. These platforms provide developers with robust environments and a variety of tools to create innovative and user-friendly apps.

The development of mobile applications often involves using third-party libraries. These libraries offer pre-built functionalities that developers can integrate into their applications, saving time and enhancing efficiency. TPLs cover a wide range of functionalities, from user interface components to complex data processing modules. By utilizing these pre-existing components, developers can streamline the app creation process, reduce development time, and ensure more robust and feature-rich applications.

However, the integration of TPLs introduces various challenges. While they significantly speed up the development process and provide essential functionalities, TPLs can also become vectors for security vulnerabilities. Additionally, managing dependencies and ensuring compatibility between various library versions can be complex and time-consuming. This complexity is further compounded by the need to keep track of updates and patches for these libraries, which are crucial for maintaining the security and stability of the application.

1.2 Motivation

The proliferation of TPLs has led to the development of numerous tools designed to detect and manage these libraries within mobile applications. Despite the availability of various tools, developers often face difficulties in choosing the most suitable one for their needs. Each tool comes with its own set of strengths and weaknesses, and the choice of tool can significantly impact the effectiveness

of TPL management. This includes critical tasks such as security assessments, compliance checks, and maintenance efforts.

The primary motivation behind this thesis is to address the gap in comprehensive, empirical comparisons of TPL detection tools for Android and iOS. With the growing number of applications and the increasing reliance on TPLs, it becomes imperative to have a detailed understanding of how these tools function. Developers often encounter challenges such as detecting outdated or vulnerable libraries, managing complex dependency chains, and ensuring that the integrated libraries do not introduce new security risks. A thorough comparison of available tools can significantly aid in mitigating these challenges by providing a clear picture of each tool's capabilities.

Another motivating factor is the necessity to highlight the differences and similarities in TPL usage between Android and iOS platforms. Each platform has its own unique set of tools, development environments, and operational characteristics. Understanding how TPL detection tools perform on each platform can provide valuable insights into platform-specific challenges and best practices. This knowledge can help developers make more informed decisions and adopt strategies that are tailored to the strengths and limitations of each platform.

Furthermore, the rapid pace of mobile technology development means that TPL detection tools must continually evolve to keep up with new challenges. This thesis aims to contribute to this ongoing evolution by not only comparing existing tools but also identifying areas where current solutions fall short. By doing so, it seeks to pave the way for the development of more robust, efficient, and user-friendly TPL detection tools in the future.

1.3 Objectives

The main objective of this thesis is to provide a detailed evaluation and comparison of TPL detection tools for Android and iOS. The specific objectives are multifaceted, aiming to offer both a practical guide for developers and a deeper understanding of the tools' capabilities and limitations.

1. **Creation of a Comprehensive Guide**: One of the primary goals is to develop a comprehensive guide that outlines the existing TPL detection tools for both Android and iOS. This guide will provide detailed information on each tool's features, installation process, and usage instructions. By doing so, it will serve as a resource for developers, helping them make informed decisions about which tools best suit their needs.

2. **Empirical Comparison of Tools**: Conducting a series of experiments to empirically compare the performance of TPL detection tools is another key objective. This involves evaluating the tools based on metrics such as precision, recall, efficiency, and resilience against code obfuscation. The empirical data collected through these experiments will help identify the strengths and weaknesses of each tool, providing insights into their suitability for different use cases. By comparing these metrics, the thesis aims to highlight

the trade-offs and advantages of each tool assisting developers in selecting the most appropriate tool for their specific requirements.

3. **Comparison of TPL Usage Across Platforms**: Examining the differences in TPL usage between Android and iOS applications is another crucial objective. This involves analyzing the same set of applications on both platforms to identify any discrepancies in TPL integration and performance. By understanding these differences, the thesis aims to provide insights into cross-platform development practices and the potential impact of TPLs on the overall performance and security of mobile applications.
4. **User Experience Evaluation**: Evaluating the usability of these tools is essential to ensure they meet the practical needs of the development community. This objective involves conducting a user experience study with developers, assessing the ease of use and interpretability of the results provided by each tool. The feedback collected from developers will provide qualitative insights into the strengths and weaknesses of each tool from a usability perspective.

By achieving these objectives, this thesis aims to provide a valuable resource for developers, helping them navigate the complex landscape of TPL detection in mobile application development. The insights gained from this research will not only aid developers in selecting the most appropriate tools but also contribute to the ongoing improvement and innovation in the field of TPL detection.

Chapter 2

Background on Mobile App Development

The landscape of mobile app development is defined by the ongoing evolution of two major operating systems: iOS and Android. These platforms underpin the vast ecosystem of mobile applications, continually shaping user experiences and technological advancements. As the basis of modern digital interaction, both iOS and Android offer developers a comprehensive suite of tools, languages, and frameworks, facilitating the creation of diverse applications to be used in a wide range of user needs.

In this chapter, we will provide a broad overview of iOS and Android development, which is essential for a deeper understanding of the discussions that follow.

2.1 iOS

Apple's iOS [1] platform continues to be a major player in the mobile operating system landscape. As of 2023, Apple boasts over two billion active devices worldwide, highlighting the extensive reach and influence of its ecosystem [2]. This ecosystem is supported by a strong user base, with more than 1.334 billion iPhone users globally [3], demonstrating significant growth and engagement across various demographics.

The App Store is the exclusive platform for downloading apps on iOS devices, and it plays a pivotal role in the iOS ecosystem. As of 2023, the App Store boasts approximately 1.8 million apps [4]. This vast library of applications underscores the App Store's critical function in facilitating a wide range of experiences for users and offering developers a significant marketplace for their apps.

2.1.1 Development Environment and Tools

The development environment for iOS applications has evolved significantly, with major advancements in programming languages and tools shaping its cur-

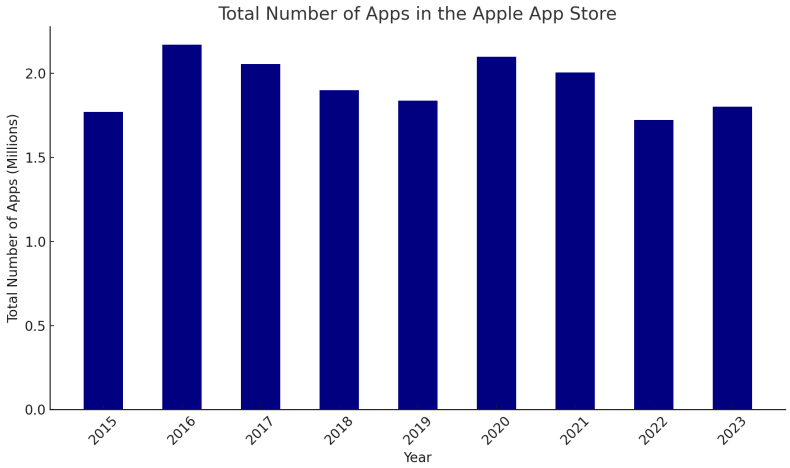


Figure 2.1: Number of iOS apps over the years.

rent landscape. Initially, iOS apps were developed using Objective-C [5], a language known for its robustness and inherited from NeXT since the iPhone's debut in 2007.

A major shift occurred in 2014 when Apple introduced Swift [6], a language designed to be more efficient, safer, and easier to use than Objective-C. Swift's modern features such as type inference and powerful error handling quickly made it the preferred choice among developers for iOS app development.

Xcode [7], Apple's integrated development environment (IDE), is central to iOS app development for both Swift and Objective-C. It offers a comprehensive toolkit that includes a source editor, a UI editor, debugging tools, and version control integration, facilitating a streamlined workflow for developers from coding to deployment.

2.1.2 Structure of Binaries

iOS apps are primarily developed using Apple's Xcode IDE, which manages the compilation, linking, and packaging of apps. The build process results in Mach-O executable files that include the app's code, dynamically linked libraries, and vendored libraries, where the latter refers to open-source libraries integrated directly into the app's source base.

Dynamically linked libraries in iOS are usually distributed as Frameworks, which contain the library code in a Mach-O executable along with necessary resources. These Frameworks make identification easier due to their naming conventions, although the library names and versions might not always be explicitly stated. Conversely, the binary code from statically linked and vendored libraries is included directly into the main app executable.

iOS apps are packaged as IPA files, essentially ZIP files with a structured format that includes a Payload folder containing all necessary runtime files, except system libraries. Each app must be digitally signed, as unsigned apps will not run

on iOS due to security policies.

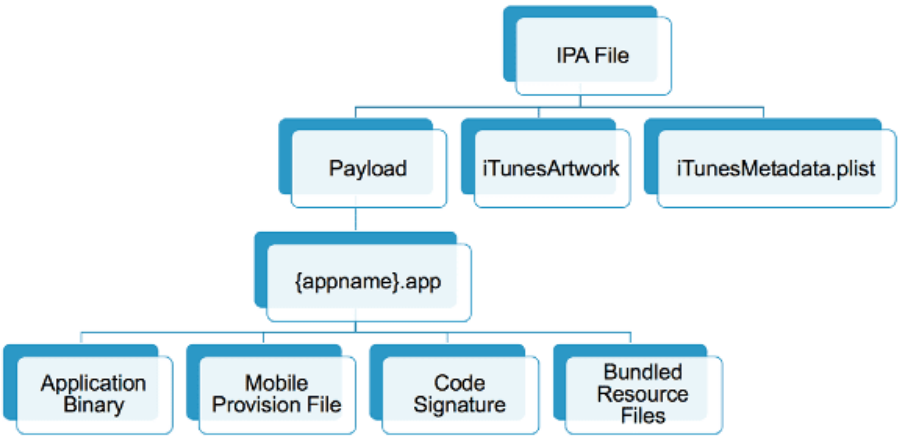


Figure 2.2: iOS Binary Structure.

2.1.3 Security Features

iOS app security incorporates a variety of techniques to safeguard apps from reverse engineering and malicious attacks. One key strategy is code obfuscation, which modifies the app's code to make it difficult for attackers to analyze and understand. While obfuscation protects an app's intellectual property and sensitive data by concealing its logic and functionality, it can also inadvertently complicate the detection of TPLs, making it challenging to identify and assess the security of embedded components.

Despite its benefits, obfuscation is not widely used across all iOS apps. A study by Wang et al. [8] revealed that only 0.06% of apps in the iTunes Store employ obfuscation, indicating a significant gap in the adoption of this security practice. This suggests that while obfuscation is a valuable tool for enhancing app security, it remains underutilized, possibly due to the complexities involved in its implementation or a lack of awareness about its advantages.

Alongside obfuscation, iOS developers also implement runtime protection mechanisms and adhere to security standards such as GDPR and PCI to ensure robust security throughout the app's lifecycle. These measures help maintaining the integrity and security of iOS applications against cyber threats.

2.2 Android

Android [9], developed by Google and initially released in 2008, has grown to become the most popular mobile operating system globally. As of 2023, there are over 3.3 billion active Android users worldwide [10], making it a dominant force in the mobile ecosystem. This open-source platform is favored for its flexibility, allowing extensive customization and integration capabilities, which have greatly contributed to its widespread adoption across various device manufacturers.

Android apps are predominantly distributed through the Google Play Store, which hosts millions of apps and continues to grow each year. Unlike iOS, Android allows the installation of apps from third-party sources, providing a less restrictive environment that appeals to both developers and users. However, this also introduces varied security challenges, which have been addressed over the years through continuous improvements in Android's security architecture.

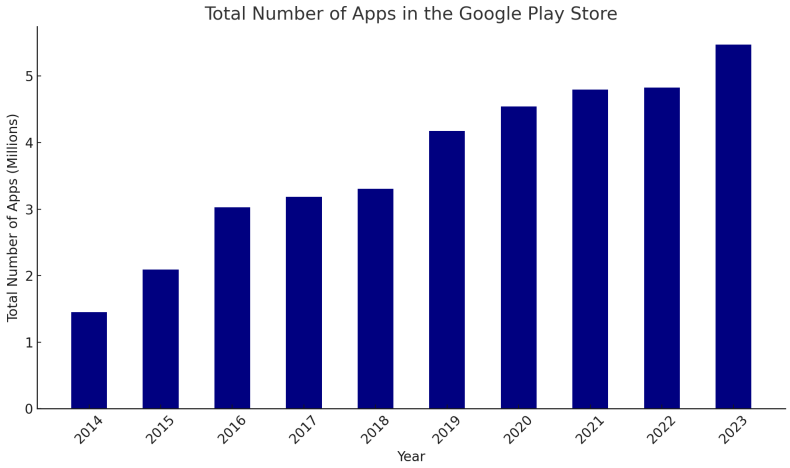


Figure 2.3: Number of Android apps in Google Play over the years.

2.2.1 Development Environment and Tools

Android application development has evolved significantly over the years, marked notably by the adoption of new programming languages that enhance development efficiency and software robustness. Initially, Android apps were primarily developed in Java, a language known for its portability, extensive libraries, and mature ecosystem. Java has been instrumental in shaping the standard for Android development due to its object-oriented structure and widespread familiarity among developers.

In 2017, Google introduced Kotlin [11] as an official language for Android development. Kotlin has since gained popularity for its concise syntax, safety features, and interoperability with Java. It reduces boilerplate code, which simplifies the creation of robust applications and makes Android development more approachable and efficient.

Android developers also occasionally use the Android Native Development Kit (NDK) when they need to implement critical parts of their applications in native-code languages like C or C++. This approach is typically reserved for performance-sensitive parts of applications, such as physics calculations or signal processing.

2.2.2 Structure of Binaries

APK files, the primary format for Android applications, is analogous to IPA on iOS based devices. To create an APK file, an Android application is first compiled

using a development tool like Android Studio or Visual Studio. Following compilation, all components of the application—including code (.dex files), resources, assets, certificates, and the manifest file—are packaged into one container file.

Like many file formats, APK files can have any name, but they must end with the ".apk" extension to be recognized by the system. This container is essentially a ZIP file that organizes its contents into a structured directory format, facilitating the distribution and installation of Android applications.

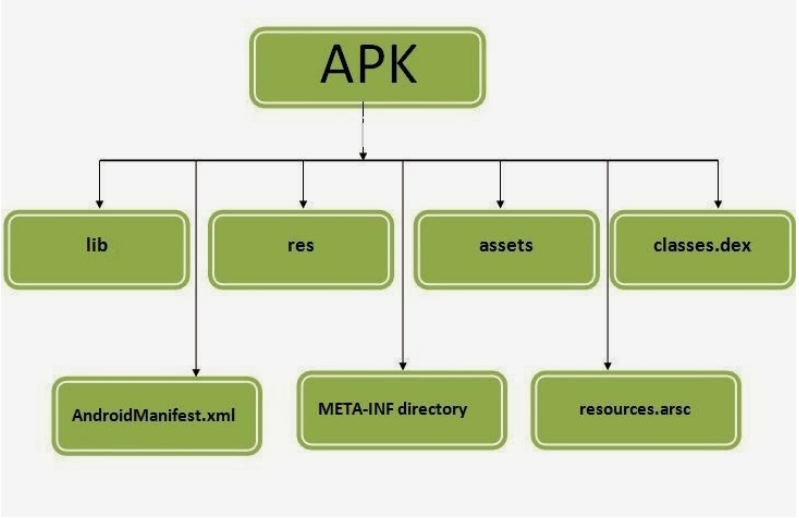


Figure 2.4: Android Binary Structure.

2.2.3 Security Features

Android's open ecosystem allows for flexibility in app distribution and development, but also introduces unique security challenges. To protect applications and maintain user security, Android and app developers employ several robust security features:

Code Obfuscation is the main technique used to protect Android apps from reverse engineering. By transforming the app's source code into a format that is difficult to read and understand, obfuscation helps secure intellectual property and sensitive data within the app. However, similar to iOS, the use of obfuscation can also complicate the detection of TPLs, posing challenges in identifying and assessing the security of embedded components. A 2018 study [12] found that approximately 25% of Android apps had implemented obfuscation greatly surpassing the 0.06% from iOS.

Chapter 3

State of the Art on Third-party Library Detection Tools

3.1 Overview of Libraries in Mobile App Development

3.1.1 Overview of Libraries in Mobile App Development

In the context of mobile app development, libraries are reusable sets of code or functions that developers can include in their applications to extend functionality without having to write the code from scratch. These libraries can include anything from simple helper functions to complex systems that manage database connections, user interfaces, or network requests. The primary purpose of using libraries is to streamline the development process, promote code reuse, and enhance application performance and maintainability.

- **First-Party Libraries** are those developed by the developers of the application itself or by the provider of the development platform, such as Google for Android or Apple for iOS. These libraries are designed to be integrated seamlessly with the platform and are often optimized for performance and security. They usually provide fundamental functionalities that are essential for the running of applications on those specific platforms.
- **Third-Party Libraries (TPLs)**, on the other hand, are developed by external entities and aren't part of the official development platform. These libraries offer specialized functionalities that may not be available through first-party libraries, such as image processing, advanced data visualization, or third-party network services integration. While TPLs can significantly accelerate development and add rich features to an app, they also introduce dependencies and potential security risks, as the app becomes reliant on code that isn't controlled by the original app developers.

3.1.2 Third-Party Libraries in iOS

iOS developers often manage their application's dependencies, including TPLs, using package managers (PMs). There are three primary PMs available for iOS:

CocoaPods, Carthage, and Swift Package Manager . Each of these supports the integration of libraries whether distributed as source code or as pre-compiled binaries.

A notable distinction among these PMs is the approach to library availability. CocoaPods utilizes a central repository to store library specifications known as Podspecs, which greatly facilitates visibility and access for developers seeking useful libraries. In contrast, Carthage and Swift PM don't operate a central repository, requiring developers to seek out libraries independently.

For library distribution via CocoaPods, developers submit a Podspec file to the public repository when they wish to publish a new library or update an existing one. CocoaPods enforces semantic versioning (MAJOR.MINOR.PATCH) for managing library versions. For example, the Podspec for Firebase, a popular library by Google, includes general information like the library's name, version, and the source URL. It also details dependencies in terms of modules, or subspecs as termed by CocoaPods. For instance, Firebase's Core subspec, which is installed by default, depends on FirebaseAnalytics and FirebaseCore, whereas the AdMob subspec additionally depends on the Google-Mobile-Ads-SDK.

To integrate these libraries into an iOS app, developers create a Podfile that specifies the required libraries. CocoaPods processes this Podfile to automatically download and incorporate the libraries into the app's Xcode project, ensuring they are compiled and linked during the app build. This results in the app containing multiple Mach-O binaries—one for the app itself and one for each library. If not specified otherwise in the Podfile, only default subspecs of libraries, such as Firebase/Core, are included.

The CocoaPods installation also generates a Podfile.lock file, which resolves and locks down the exact versions of the libraries used, based on the constraints specified in the Podfile. For example, a Podfile might result in Firebase 4.7.0, Objection 1.6.1, and SSZipArchive 2.2.3 being included in the project. However, it's important to note that the Podfile.lock file isn't included in the built app and is only available when building the app from source code. This limitation means that the Podfile.lock cannot be used for TPL detection when only the app's binary code is available.

```
  platform :ios, '10.0'
  use_frameworks!
target 'StopAndSearch' do
  pod 'Gloss'
  pod 'Alamofire', '~> 4.3'
  pod 'ChameleonFramework/Swift', :git => 'https://github.com/ViccAlexander/Chameleon.git'
  pod 'Charts', '~> 3.0.1'
  pod "PromiseKit/Alamofire", "~> 4.0"
  pod 'RSClipperWrapper', :git => 'https://github.com/rusty1s/RSClipperWrapper.git'
end
```

Figure 3.1: Podfile content of an app with 6 TPLs.

3.1.3 Third-Party Libraries in Android

Android developers commonly utilize package managers to manage their application's dependencies, including TPLs. The most widely used package managers in the Android ecosystem are Gradle and, to a lesser extent, Apache Maven. These tools facilitate the integration of libraries, whether they are distributed as source code or pre-compiled binaries.

Gradle, which is integrated within Android Studio, uses a flexible dependency declaration approach. Developers specify dependencies in the *build.gradle* file of their application, where they can list TPLs with their respective versions. Gradle connects to repositories like Maven Central or JitPack to retrieve these libraries. Unlike CocoaPods in the iOS environment, Gradle doesn't rely on a central repository but pulls libraries from various sources as configured by the developer.

When publishing a library, Android developers typically host their library on a public Maven repository and provide a Gradle dependency snippet that users can add to their build.gradle files. This snippet specifies the library's coordinates (group ID, artifact ID, version) within the repository. For example, a popular networking library might be included with a line such as implementation 'com.squareup.retrofit2:retrofit:2.9.0', where retrofit is the library, and 2.9.0 is the version.

This build configuration is processed by Gradle during the app's build process, where it resolves and downloads the specified libraries, integrating them into the app's APK file. Each library and its dependencies are compiled into DEX files, which are included in the final application package.

Like iOS, the final build artifact (APK) doesn't contain details like the *build.gradle* configuration file, making it challenging to detect and analyze TPLs directly from the app's binary code without additional context or source code access.

```
buildscript {
    repositories {
        mavenLocal()
        jcenter()
        google()
    }
    dependencies {
        classpath 'com.android.tools.build:gradle:7.4.1'
        classpath 'com.github.axet:gradle-android-dx:0.3.2'
    }
}
```

Figure 3.2: build.gradle content of an app with 2 TPLs.

3.2 Third-Party Library Detection Techniques

While the fundamental detection process generally follows a consistent sequence of steps across various platforms such as iOS and Android, not all detection tools employ each step uniformly. Here's an overview of the general steps involved in TPL detection based on the tools we have reviewed:

- **Step 1: Preprocessing** The process begins with the preprocessing of mobile applications, where apps are decompiled using reverse-engineering tools to revert the compiled binaries into a more human-readable intermediate representation (IR). This IR is very important as it facilitates the application of different analytical techniques in subsequent steps.
- **Step 2: Library Instance Construction** This step involves segmenting the app into distinct modules: the primary module, containing code from the host app, and non-primary modules, which may consist of code from one or more TPLs. The aim is to isolate TPLs from the host app's native code and identify the boundaries of each TPL candidate. Notably, not all detection tools perform this step. Recent tools like LibScan for Android and LibKit for iOS opt to bypass this module decoupling process. Reasons for this may include the significant computational overhead it introduces, or because these tools can achieve sufficient accuracy and efficiency without explicitly separating code into modules. This variability in approach suggests that while library instance construction can enhance detection granularity, it may not always be necessary or justified given the trade-offs in performance and complexity.
- **Step 3: Feature Extraction** Following the potential isolation of TPLs, this step involves extracting features that can uniquely represent these libraries. Common features extracted include API calls, control flow graphs, and unique method signatures, chosen for their ability to effectively distinguish between different libraries.
- **Step 4: Library Instance Identification** The final step in TPL detection involves identifying the library instances within the app using one of two main methods:
  - Clustering-based methods: These techniques involve grouping non-primary modules that exhibit similar features into clusters. Each cluster is considered to represent a distinct TPL. This approach is particularly useful for discerning patterns and commonalities among different modules, facilitating the identification of TPLs based on shared characteristics.
  - Similarity comparison methods: These methods involve comparing extracted features of an app's modules against known TPL features. There are variations within this approach: Some tools, such as LibKit, use a pre-built database of TPL signatures to compare against. Others, like LibScan, calculate the signatures in real time to determine if a specific TPL is present in the binary.

Both approaches offer distinct advantages and cater to different aspects of TPL detection. A more in-depth discussion of these methods, including the specifics of tools like LibKit and LibScan, will be provided in subsequent sections to explore how they implement these techniques.

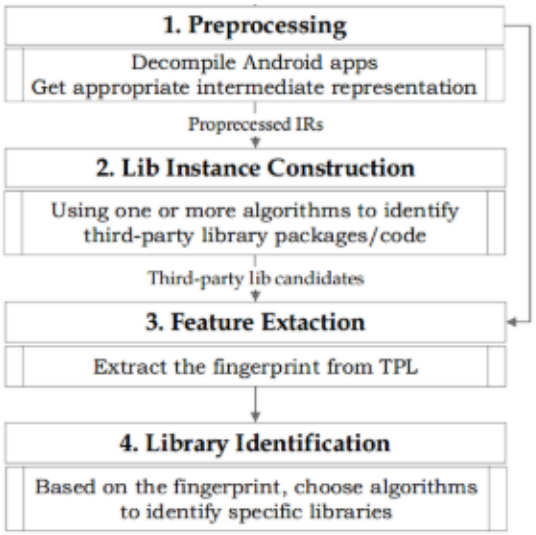


Figure 3.3: Typical process of TPL detection.

3.3 iOS Detection Tools

3.3.1 CRiOS

CRiOS [13] developed by Orikogbo et al. in 2016 has refined the process of identifying TPLs in iOS applications through a detailed and methodical two-phase approach, utilizing large datasets of applications rather than relying on pre-compiled lists. This strategy addresses the limitations of other methods by directly analyzing application binaries to discern developer-written classes from those belonging to TPLs.

Library Detection Methodology

The identification process begins once the .ipa file of an application is downloaded from Apple's App Store. CRiOS employs a tool called class-dump to extract a header file for every class in the application's binary. These header files provide essential data, including class names, inheritance chains, defined methods, and categories. This comprehensive class information establishes a one-to-many relationship between the application and the classes it utilizes, forming the foundational dataset for library detection.

Tracking each class across different applications is the next critical step. Intuitively, classes developed by the application's developers tend to appear only within a single application, whereas third-party library classes are likely to be found across multiple applications. The more applications a class appears in, the greater the likelihood that it is part of a library. CRiOS consolidates this

data to identify which classes are used by multiple applications, indicating their probable library origins.

Two-Phase Clustering Approach

- **Phase 1, Initial Clustering Based on Class Prefixes:** In the first phase of clustering, CRiOS analyzes the prefixes of class names, which often indicate their library origins. Classes are initially grouped into singleton clusters, which are then potentially merged based on their name prefixes. Mergers occur in two scenarios:
  - Scenario A: All classes in both clusters share a common prefix of at least a predefined length composed exclusively of uppercase letters, capitalizing on naming conventions seen in well-known libraries like the Facebook SDK (prefix 'FB') or the GPUImage library (prefix 'GPU').
  - Scenario B: All classes share a longer common prefix that may include lowercase letters. This scenario accommodates libraries like Flurry, which uses a mixed-case prefix. This step avoids incorrectly merging unrelated classes that merely share a common prefix, ensuring that only genuinely related classes are grouped.
- **Phase 2, Refining Clusters Based on Class Cohesion:** The second phase focuses on the cohesion among classes within these clusters. Cohesion is measured by the interaction between methods of different classes. CRiOS merges two clusters if the average of the highest cohesions between classes surpasses a predefined threshold. A penalty factor is applied depending on the length of the common prefix shared between classes, giving greater weight to cohesion among classes with common prefixes, as these are more likely to be from the same library.

This cohesion metric ensures that the clusters reflect functional groupings within the libraries, rather than mere syntactic similarities. By assessing the interaction patterns between classes, CRiOS can more accurately identify and group classes that operate as part of the same library, even distinguishing between different versions of the same library based on their interaction patterns.

Although CRiOS effectively clusters potential library components within iOS applications, it doesn't specify the exact libraries due to the absence of a comparison against a known library dataset. To fully identify these libraries, integrating CRiOS with a comprehensive library database would be essential. Additionally, the unavailability of CRiOS's source code limits our ability to test and compare its efficacy with other tools.

3.3.2 iLibScope

iLibScope [14], developed by Guo et al. in 2017, incorporates a structured four-step process to analyze iOS applications. This system is designed to build detailed profiles for libraries and applications, index these profiles for efficient searching, recover instances of libraries within the applications, and accurately match these instances to their corresponding library versions.

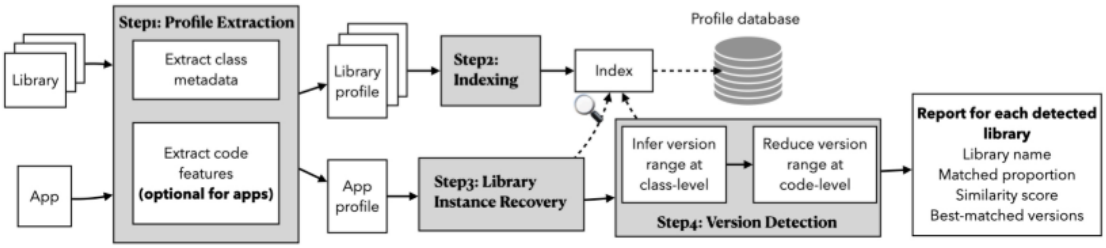


Figure 3.4: Overall architecture of iLibScope.

Step 1: Profile Building for Libraries and Applications

The foundational step in iLibScope's process involves constructing detailed profiles for each version of the libraries and the target iOS application. These profiles are structured on two levels to balance quick preliminary scanning with in-depth analysis:

- **Class-Level Profile**: This profile is generated from the class metadata, which is readily accessible due to Objective-C's retention of class metadata to support runtime operations. The profile includes class names, inheritance information, and method definitions. Despite being less detailed, class-level profiles are quick to build and provide a snapshot useful for initial filtering.
- **Code-Level Profile**: Constructed after an exhaustive semantic analysis of the application's binary code, this profile includes features extracted from the assembly code analyzed by IDA Pro. The code-level profile incorporates callee information from method bodies, class references, selector references, constant strings, and external symbols. Although building this profile is time-intensive, it captures the variations between different library versions, crucial for precise version identification.

Step 2: Indexing Library Profiles

Once the profiles are established, iLibScope builds an index of all library profiles to streamline the search process during the detection phase. Each class and method defined within the profiles is indexed by name, allowing for rapid retrieval based on specific library characteristics. This indexing is very important for enhancing the system's efficiency, enabling quick comparisons between the indexed library data and the classes found within the target application.

Step 3: Library Instance Recovery

Recovering library instances from the application involves a sophisticated matching process at the class level. iLibScope organizes the app's classes into a structured graph to manage and resolve the complexities introduced by code duplication among libraries:

- **Initial Classification**: Classes are initially categorized into a graph where nodes represent potential library matches. Classes with ambiguous origins

may float between multiple nodes, reflecting uncertainty in their library affiliation.

- **Resolution of Ambiguities**: The system processes each class by comparing its profile against those of potential library matches. Classes are then either confirmed in their preliminary node or shifted to another based on a higher similarity score or better contextual fit, thereby resolving ambiguities and refining the detection of library instances.

Step 4: Version Matching

The final step in iLibScope's process is to determine the most accurate library version for each identified instance. This is achieved by utilizing the detailed code-level profiles:

- **Fine-Grained Analysis**: The system narrows down potential versions by analyzing the specific code features associated with each library instance, focusing on the distinct differences captured in the code-level profile.
- **Best Match Determination**: Utilizing algorithms such as a variant of the Manhattan distance for feature comparison, iLibScope evaluates the similarity scores between the features of the library instance and those of different versions, selecting the version with the highest score as the best match.

3.3.3 LibKit

LibKit [15], developed by Domínguez-Álvarez et al. in 2023, utilizes a fingerprint-based identification system specifically tailored for iOS applications. This advanced technique comprises two main phases: library fingerprint generation and library detection, designed to accurately identify TPLs within iOS apps.

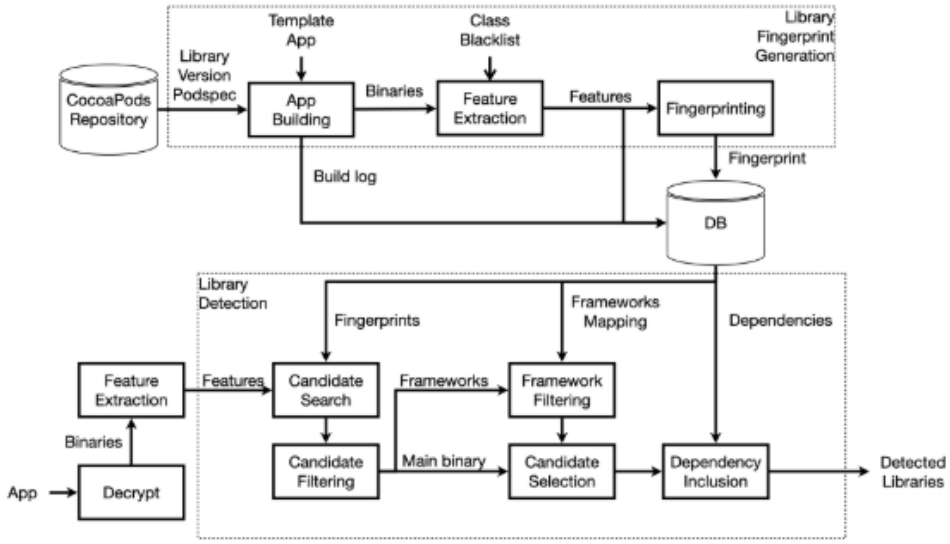


Figure 3.5: Overall architecture of LibKit, illustrating its two-phase structure.

Phase 1: Library Fingerprint Generation The process begins with the generation of library fingerprints, which involves several key steps:

- **Step 1: Collection and Building of Libraries**: LibKit automatically collects libraries from the CocoaPods repository. It retrieves the library's source code and all dependencies recursively, building them into a template application that acts as a container solely for the library. This process ensures the generation of executable Mach-O binary files necessary for further analysis.
- **Step 2: Feature Extraction**: Using a lightweight static analysis, LibKit extracts syntactic code features from each Mach-O binary file. These features are carefully selected to avoid inclusion of any that might degrade the fingerprint quality, employing a blacklist for this purpose. The tool utilizes the simhash technique, which is adept at generating similar fingerprints for similar objects, thereby allowing the system to recognize minor version differences within the same library more effectively than major ones.
- **Step 3: Fingerprint Storage**: Each extracted feature set is transformed into a unique fingerprint representing the library. These fingerprints are then stored in a database, forming a comprehensive repository of known TPL fingerprints. The database is used in the subsequent phase to identify libraries in new iOS applications.

Phase 2: Library Detection

- **Step 1: Decryption and Parsing** Initially, LibKit decrypts the iOS application package (IPA), addressing Apple's encryption protocols that protect application code. Following decryption, the system parses each binary to extract structural features, specifically focusing on class and method characteristics. This foundational data extraction sets the stage for the subsequent fingerprinting phase.
- **Step 2: Fingerprint Generation and Matching** Using the parsed data, LibKit generates fingerprints for each class within the binaries. These fingerprints are crafted using the Sim-hash algorithm, which effectively produces comparable hashes for similar features, allowing for minor variations due to code changes between library versions. The choice of Simhash is strategic for its ability to reduce data complexity while maintaining the ability to differentiate between closely related versions.
- **Step 3: Candidate Search and Matching** The core of LibKit's detection algorithm is its candidate search function. This function identifies potential library matches by comparing each generated fingerprint against a database of known library fingerprints. Matching is quantified using a similarity measure based on the Hamming distance between Simhash values, which is then normalized against a similarity threshold. This threshold is important for distinguishing between actual matches and near misses, facilitating a high level of precision in identifying library versions.
- **Step 4: Filtering and Final Selection** Once potential matches are identi-

fied, LibKit undertakes a filtering process to refine these candidates. This involves assessing the proportion of matched features to the total features within the library, ensuring that only candidates with a significant match ratio are retained. The system differentiates between main app binaries and Frameworks, applying tailored filtering logic to each. For Frameworks, the process is more direct as their names provide clear indicators of the libraries they contain.

For the main app binary, which could contain multiple libraries, LibKit applies a set cover algorithm to select the most appropriate matches. It ranks library candidates based on a composite score that factors in the coverage and similarity of matched classes as well as the library's prevalence in the candidate set.

- **Step 5: Dependency inclusion** Finally, for each selected library, LibKit reports all its dependencies in the final list of libraries found in the app, since these are implicitly included in the bundle.

The library fingerprint database generation in LibKit is carried out only once for each library version in the CocoaPods repository, establishing a database that doesn't require regeneration unless new library versions are added. Additionally, the source code for LibKit is available, allowing us to test and verify the tool's later on.

3.4 Android Detection Tools

3.4.1 AdDetect

Developed by Annamalai Narayanan et al. in 2014, AdDetect [16] is one of the pioneering Android tools focused specifically on identifying third-party advertising libraries. It employs machine learning techniques to analyze and categorize app components, enhancing the automation of ad library detection within Android applications.

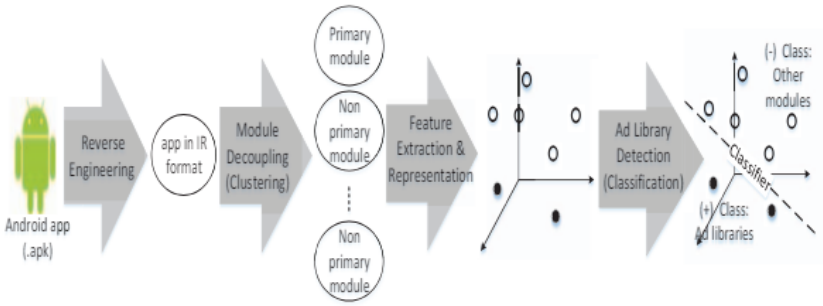


Figure 3.6: Overall architecture of AdDetect.

Step 1: Reverse Engineering

- **Decompilation and Analysis**: AdDetect starts by decompiling the Android APKs to transform .dex files, which are designed to run on the Dalvik Vir-

tual Machine (DVM), back into smali code—a popular intermediate representation (IR). This decompilation is facilitated by tools like apktool [17], which also helps in extracting the architecture information, such as packages present in apps. The retention of rich semantic information in .dex files enables sophisticated reverse engineering at both the IR and Java code levels.

Step 2: Module Decoupling

- **Construction of Package Dependency Graph (PDG)**: Based on the architectural information gleaned from the reverse engineering phase, a PDG is constructed to visualize the dependencies and interactions between various components of the application.
- **Application of Hierarchical Agglomerative Clustering (HAC)**: HAC is applied to the PDG to segregate the app into distinct modules. This segmentation isolates the core functionalities of the app contained in the primary module from other functionalities, such as ad libraries, social SDKs, and analytics libraries.

Step 3: Feature Extraction and Representation

- **Semantic Feature Extraction**: AdDetect extracts semantic features from each decoupled module. These features include the usage of Android components, permissions, and APIs—elements that are indicative of the functionality and potential behaviors of the modules.
- **Vector Representation in Feature Space**: The extracted features are quantified and represented as vectors in a feature space. This standardized representation facilitates the application of machine learning techniques by converting qualitative attributes into a format that can be algorithmically analyzed.

Step 4: Ad Library Detection

- **Classifier Training and Application**: With the feature vectors prepared, AdDetect constructs a machine learning classifier. This classifier is specifically trained to recognize patterns and features that correspond to advertising libraries. Once trained, the classifier is used to perform automated semantic detection of ad libraries within the apps.
- **Automated Detection Process**: The classifier systematically evaluates each module's feature vector to determine whether it matches the characteristics of known ad libraries. This step is key as it automates the detection process, enabling AdDetect to scale across multiple applications effectively.

While AdDetect represents a significant advancement in identifying libraries within Android apps, its specialized focus on ad detection is a notable limitation. Due to this specificity, it will not be included in future comparisons with more comprehensive third-party library detection tools that analyze a broader range of library types.

3.4.2 LibRadar

Developed by Ziang Ma et al. in 2016, LibRadar [18] operates through a two-phase process, initially building a detailed feature database from over a million apps and subsequently using this database to quickly identify third-party libraries in new applications.

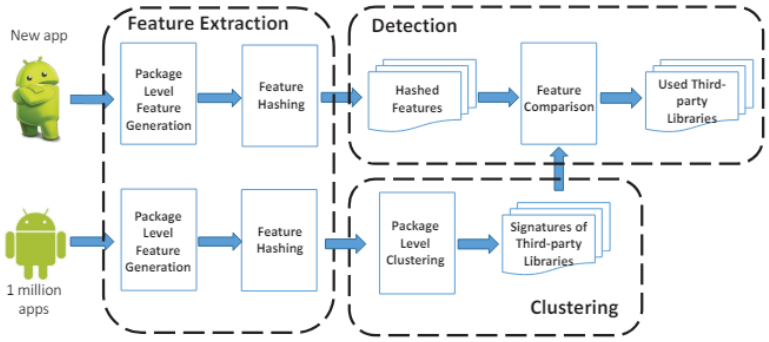


Figure 3.7: Overall architecture of LibRadar.

Phase 1: Database Creation

Phase one of LibRadar's operation involves a one-time, analysis and database creation process that serves as the foundation for rapid third-party library detection in phase 2.

- **Feature Extraction**
  - **Stable Code Feature Extraction**: LibRadar targets features that remain consistent even when the code is obfuscated. The chosen features are the frequencies of various Android API calls extracted from the Smali code of the apps. These features are robust as they cannot be altered by obfuscation and accurately reflect the low-level behaviors typical of third-party libraries.
  - **Feature Hashing** To manage the vast amount of data (over 1 million apps and 71 million packages), features of each package are hashed into a lightweight format. This hashing not only reduces the complexity but also speeds up the subsequent comparison processes. A unique fingerprint for each package is created by combining the hash value of feature vectors with meta features like the total number of API calls and the types of these calls. This ensures that even if different feature vectors yield the same hash value, the combination with meta features will still provide a unique identifier for each package.
- **Multi-level Clustering**
  - **Overall Clustering Process**: Features from all packages are aggregated, and these aggregated features form the basis for clustering. This process helps in identifying the largest possible libraries that are

commonly used across multiple apps, distinguishing them as distinct clusters.

- **Strict Clustering Criteria**: Only packages with identical feature sets are grouped together, ensuring that each cluster represents a unique library or a version of a library. A threshold is set to determine which clusters represent third-party libraries, based on their prevalence across the dataset.
- **Package Name Identification**: Despite the potential obfuscation of package names, LibRadar can infer the original names of packages within a cluster from those that aren't obfuscated. This is done by analyzing the frequency of package names across clusters and discarding those that are clearly obfuscated, thus retaining only meaningful and recognizable package names.

Phase 2: Library Detection in New Apps

In the second phase, LibRadar applies the database of hashed library features to new Android apps to detect third-party libraries efficiently.

- **Feature Matching**: When a new app is analyzed, LibRadar extracts its features in the same manner as the database creation phase. It then quickly compares these features against its database of third-party library signatures using feature hashing.
- **Instant Detection**: Due to the pre-hashed and indexed nature of the database, the comparison process is rapid, allowing for the near-instant detection of third-party libraries within any given app.

The approach introduced by LibRadar, combining clustering techniques and feature hashing for third-party library detection, offers a potentially effective solution for analyzing Android applications. Given that LibRadar is available for public use, it enables us to conduct comparative tests with other detection tools later on to evaluate its efficacy and accuracy comprehensively.

3.4.3 LibScout

The workflow of LibScout [19] approach consists on two distinct phases. Initially, profiles are generated from the class hierarchy information of each library version in the repository, ensuring robustness against code obfuscation techniques such as control-flow randomization and API hiding. This focus on class hierarchy rather than concrete library code is essential for reliable detection. Following this, the tool statically detects libraries within applications by extracting app profiles and employing a matching algorithm. This algorithm assesses the profiles and assigns a similarity score between 0 and 1, which indicates whether a library version was matched exactly, partially, or not at all in a given application

Profile Extraction LibScout utilizes a streamlined approach for profile extraction employing a simplified Merkle tree structure, which focuses on packages,

classes, and methods. In this structure, each package directly connects to the root to facilitate efficient comparisons between libraries and applications. The organization into three layers allows the structure to simplify the matching process by handling each distinct package as a direct child of the root.

To tackle the issue of code obfuscation such as API hiding, LibScout strategically uses only the essential components of method signatures—argument lists and return types—for hashing. This approach focuses on these elements because they are less susceptible to obfuscation compared to method names and other modifiers, thereby enhancing the robustness of the profiles against various obfuscation techniques.

Additionally, before constructing the hash tree, LibScout performs byte-code normalization to eliminate compiler-generated artifacts. This step ensures that the profiles are based purely on developer-written code, omitting any extraneous elements introduced during the compilation process. By focusing on the inherent functionality of the code, LibScout ensures that its profiles are accurate and resistant to common obfuscation tactics

Profile Matching

In LibScout, profile matching between library profiles and app profiles is performed using a modified Merkle tree where the comparison is conducted top-down. Each non-leaf node in these trees is labeled with the hash of its child nodes, making it possible to efficiently and precisely determine if a library's profile matches with an application's profile. This approach provides a framework to compute a similarity score ranging from zero to one, indicating the extent to which a library version matches an application, either exactly, partially, or not at all.

Initial Matching:

The matching process starts by verifying if all library package hashes are included in the app's hash tree. An exact match is indicated when the similarity score is one, suggesting that every package hash from the library can be found in the application. Partial matches occur when only a subset of the library's package hashes aligns with those in the application, commonly due to the application including a version of the library that is different from those in the database or only incorporating parts of a library through processes like dead-code elimination.

Sophisticated Matching Techniques:

If its not an exact match then sophisticated techniques are used to enhance matching accuracy, especially in cases where direct comparisons might lead to false positives due to package hierarchy discrepancies, LibScout employs a multi-step approach:

1. **Candidate List Generation:** Each library package is associated with a list of candidate app packages where at least 50% of class hashes match. These candidates are then scored and sorted based on their similarity to the library package.

2. **Package Linking:** This step involves linking library packages with candidate app packages based on matching package names. If obfuscation or renaming obscures direct matches, the algorithm seeks to identify potential root packages that might represent the library within the app.

3. **Partitioning and Structured Matching:** Instead of testing all combinations, the algorithm partitions the candidates by potential root packages and computes the maximum score for each partition. This structured approach ensures that the combinations respect the library package hierarchy, avoiding invalid matches where the relationship (parent, child, sibling) between packages doesn't align.

4. **Global Maximum Calculation:** The final similarity score is the maximum value obtained across all partitions, representing the best possible match. The algorithm sums up the matched classes and computes the final similarity score, which indicates how closely the app resembles the library at the class level.

LibScout offers a promising approach to detecting third-party libraries, particularly in scenarios involving obfuscated code. With its method focused on structured comparisons and hashed profile matching, it addresses key challenges in library detection within Android applications. As the tool is publicly available, we have the opportunity to test and compare its capabilities against other detection tools in future analyses

3.4.4 ORLIS

ORLIS [20], developed by Li Li et al. in 2016, introduces two stage detection process aimed at identifying third-party libraries within Android applications. This method is designed to efficiently handle the complexities associated with library detection, especially in cases where obfuscation techniques are employed. The first stage focuses on app-library similarity, using method string features to filter potential library matches. The second stage advances to class similarity, where detailed mappings between app classes and library classes are established, providing a deeper level of analysis and precision in identifying library usage within apps.

Feature Extraction ORLIS begins its analysis by extracting a comprehensive set of features from Android applications. These features are specifically chosen for their resilience against obfuscation. By focusing on method-level and class-level information, ORLIS captures essential aspects of the application's structure and behavior. This includes the relationships between methods and the structural organization of classes, which are crucial in understanding the underlying functionality of the application. Such depth in feature extraction ensures that ORLIS can detect libraries despite attempts to mask or alter code through obfuscation.

Two-Stage Detection Process

- **Stage 1: App-Library Similarity**

The first stage of ORLIS's detection process involves assessing the similarity between an application and libraries by analyzing method string features. For each app and library, method strings are sorted, concatenated, and transformed into a digest. This digest is then compared with pre-computed digests from the library database to identify potential library matches within the app. This effective filtering technique significantly reduces the number of libraries to be examined in the next stage, streamlining the detection process.

- **Stage 2: Class Similarity**

  In the second stage, ORLIS establishes a mapping between classes in the app and those in the libraries identified from the initial screening. It calculates a unique string feature for each class based on its method details and the hierarchy of its superclasses, excluding universal classes like java.lang.Object. Each class feature is then digested, and pairwise similarity scores between app classes and library classes are computed based on these digests.

The availability of Orlis's code enables us to thoroughly test and compare the tool against other library detection solutions, allowing for comprehensive evaluations of its effectiveness in identifying third-party libraries in Android applications.

3.4.5 LibID

Developed by Jiexin Zhang et al. in 2019, LibID [21] is designed to identify TPLs in Android effectively, even against popular obfuscation techniques. To accommodate different needs, LibID introduces two distinct identification schemes: LibID-S and LibID-A. LibID-S focuses on scalability and speed, optimizing for faster analysis, while LibID-A emphasizes accuracy, providing a more precise identification at the cost of increased processing time.

The workflow of LibID consists of two major steps: profiling and matching. Initially, LibID generates profiles for every library and app binary. These profiles are then compared in the matching process to determine the likelihood of any library's presence in an app. If a match is found, LibID also reports the in-app package names of the matching libraries, aiding researchers in quickly locating the library code within the app.

LibID-S

Step 1: Profiling

In the profiling step of LibID-S, both applications and libraries undergo a uniform process due to the application of identical techniques for both. This step allows the construction of a reliable representation of the code that is resilient against common obfuscation techniques such as identifier renaming, code shrinking, and control flow randomization.

- **Control Flow Graph (CFG) Construction:**

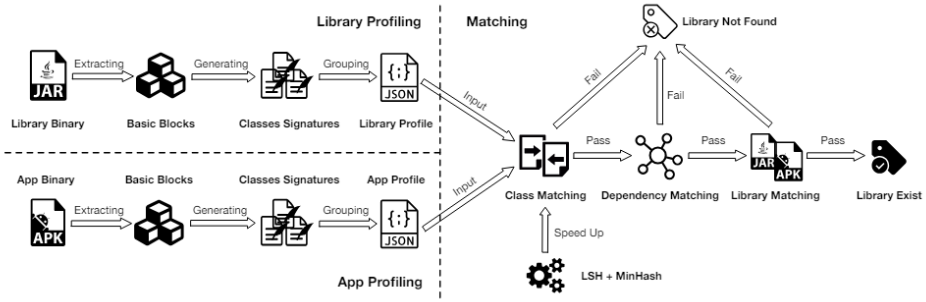


Figure 3.8: Overall architecture of LibID.

The initial phase in the profiling process involves constructing the Control Flow Graph (CFG) from the binary of a library or an app. This graph delineates the flow of control within the code, outlining various basic blocks – the smallest sequences of instructions without jumps in or out.

- **Extraction of Basic Blocks:** Following CFG construction, all basic blocks within the graph are extracted. Each block is analyzed to identify fundamental operations and interactions, such as field accesses (reads and writes) and method calls, which are very important for building the block's signature.

- **Basic Block Signatures:**

  Each basic block is then represented through a signature that captures essential operations in a manner invariant to common obfuscation techniques. These signatures include detailed records of field-related operations and method calls within the block, noting the frequency and type of each operation. For instance, multiple field reads are denoted accordingly in the signature to reflect the actual activities within the block.

- **Augmentation with Class Features:**

  To enhance the descriptive power of basic block signatures, they are prefixed with additional class-related features: Class Access Flag, Superclass Name, Class Interfaces and Method Descriptor.

These enriched basic block signatures, termed class signatures, collectively form a dictionary for each app or library. This dictionary maps the name of each class to its set of class signatures, providing a robust profile that effectively represents the structural and operational blueprint of the application or library.

Step 2: Matching

- **Class Matching:**

  This initial step determines potential matches between classes in the library and the app. Instead of directly comparing class signatures for exact matches—which may not be viable due to obfuscations like dead-code elimination—LibID-S employs a "class signature containment index." This

index measures how much of a library class's signature is contained within an app class's signature, accommodating changes that obfuscation may introduce.

- **Dependency Matching:**

  After identifying possible class matches, the next challenge is confirming these matches. Given that a library class can only correspond to one app class (and vice versa), the process involves creating a match matrix to filter out false matches. This matrix respects the hierarchical structure of classes within packages, ensuring that matches aren't only probable but also logically consistent with the structure of both the app and the library.

- **Library Matching:**

  The final step assesses the overall library match within the app. This is quantified by the library match index, which compares the proportion of matching productive classes (those contributing unique signatures) between the library and the app. This index helps determine the presence of a library even if only a subset of its classes is used due to code optimizations like shrinking.

Throughout these stages, LibID-S leverages efficient techniques like Locality-Sensitive Hashing (LSH) to speed up the matching process. LSH helps group similar items together, increasing the efficiency of finding potential matches by reducing the need for pairwise comparison. Additionally, hierarchy constraints ensure that matches respect the organizational structure of packages and classes, reducing the likelihood of incorrect matches.

LibID-S

Step 1: Profiling

In the profiling phase of LibID-A, an enhanced version of LibID-S, the focus shifts towards incorporating finer-grained features to achieve more accurate library detection. This approach addresses the shortcomings of LibID-S by utilizing detailed dependency information among classes, which significantly improves the precision in identifying library matches and class pairings within apps.

- **Dependency Recording**:

  LibID-A meticulously records interactions and relationships within the code that are susceptible to obfuscation, unlike its predecessor. This includes:

  - Method Calls: Records are made for calls where the callee might be obfuscated, noting the caller and callee class and method names within an invocation dictionary.
  - Class Inheritance: Captures the superclass of any class likely to be obfuscated, logging this information under the respective class in an inheritance dictionary.
  - Class Interfaces: Similar to inheritance, any obfuscatable interfaces implemented by a class are documented under the class's entry in an

interface dictionary.

These records are only maintained if the entities involved aren't part of the Android SDK, thereby focusing on those aspects of the code more likely to be manipulated through obfuscation techniques.

- **Dependency Graph Construction**:

  Utilizing the collected data, LibID-A constructs dependency graphs to visually and structurally represent the relationships between classes. Each node in these undirected graphs represents a class, while edges denote various types of dependencies—such as method invocations, class inheritance, or interface implementations—linking these classes.

This granular approach ensures that LibID-A doesn't rely on static identifiers like method or class names for matching, which are vulnerable to obfuscation through renaming. Instead, it builds a robust framework that captures the essence of class interactions and dependencies, providing a more resilient and accurate system for identifying libraries and class match pairs within Android applications.

Step 2: Matching

Asi in the case of LibID-S, here, the matching process enhances the precision of library identification by employing three interconnected stages: Class Matching, Dependency Matching, and Library Matching.

- **Class Matching:** Similar to LibID-S, LibID-A uses the class containment similarity index to identify potential class matches between the library and the app. However, LibID-A further accelerates this process using Locality-Sensitive Hashing Ensemble, which facilitates faster containment queries, reducing the time required for pair-wise comparisons.

- **Dependency Matching:**

  This stage builds on the basic class matching by introducing additional constraints to refine the accuracy of matches:

  - Invocation Constraints: Ensures that if a method call between two classes is present in the app, a corresponding call must exist between their matched classes in the library, reflecting the same method descriptor.

  - Inheritance Constraints: If one class is a superclass of another in the app, their matched classes in the library must replicate this inheritance structure.

  - Interface Constraints: Similar to invocation constraints, if one class is an interface of another in the app, this relationship must be mirrored between their matched classes in the library. These constraints are key for maintaining the structural integrity and functional coherence between matched classes, thereby reducing erroneous matches that

could occur due to obfuscation techniques like code shrinking or identifier renaming.

- **Library Matching:** To address complexities introduced by class repackaging and shrinking, LibID-A incorporates advanced techniques:
  - Relation Pruning: This technique ensures that only classes within the same connected component in the dependency graph are considered for matching, thereby maintaining relevance and reducing noise from unrelated classes.
  - Ghost Hunting: Identifies and excludes 'ghost classes'—classes that call methods from external libraries not present in the current library being analyzed but share the same package structure.

  LibID-A improves upon LibID-S by using detailed constraints for precise class-level matching and library version identification. It employs dependency graphs to handle class repackaging and shared-package interference effectively. Although finer-grained features slow down the process, LibID-A uses LSH Ensemble to quicken matching and minimize false positives. Its operations are fully parallelizable, enhancing performance across various apps.

3.4.6 LIBLOOM

LIBLOOM [22], developed by Huang et al. in 2023 employs a sophisticated method for detecting TPLs in Android applications, utilizing a structured approach based on the concept of set inclusion. Central to this approach is the use of Bloom filters, a space-efficient probabilistic data structure that enables quick membership checking.

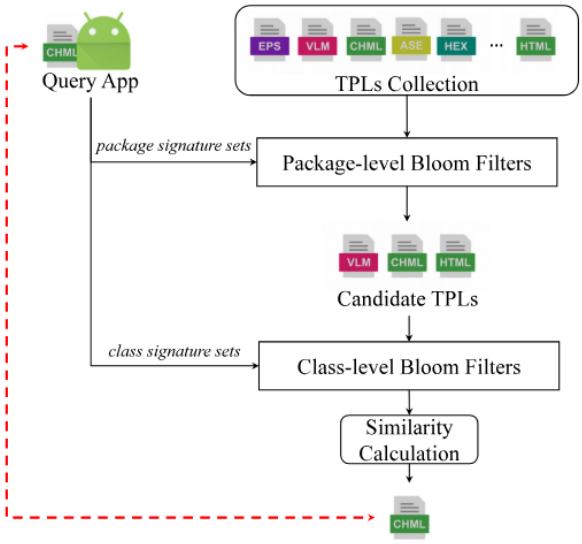


Figure 3.9: Overall architecture of LIBLOOM.

Feature Extraction

The initial phase in LIBLOOM involves the extraction of key features from both the package and class levels of Libraries, which are then used to create detailed signature sets. This process simplifies the complex attributes of the application into three main types of signatures:

- **Method Signatures:** Capturing the functionality of methods while mitigating identifier renaming through the use of fuzzy descriptors.
- **Field Signatures:** Focusing on the data types processed by a class to maintain the essential characteristics of fields while discarding variable names.
- **Hierarchy Signatures:** Outlining the structural relationships in the application by tracking class inheritance and interface implementations.

These signatures are then used to construct the Bloom filters at both the package and class levels, facilitating the subsequent stages of TPL detection.

Package-Level Bloom Filters

With signature sets prepared, LIBLOOM constructs package-level Bloom filters. These Bloom filters serve as compact probabilistic representations of the entire signature set of a package, allowing for efficient initial screening and overlap measurement between the application and potential TPLs. The process involves:

- **Bloom Filter Construction:** For each package, LIBLOOM constructs a Bloom filter by hashing each element of the package's signature set using a predetermined number of independent hash functions. The outputs of these hash functions determine the bit positions in the Bloom filter that are set to one.
- **Overlap Measurement:** LIBLOOM measures the overlap between the package-level Bloom filters of the library and the app. This is quantified by computing the ratio of common set bits to the minimum Bloom filter size, which provides a preliminary similarity score that helps in identifying promising candidate TPLs for deeper analysis.

Class-Level Bloom Filters

Following the identification of candidate packages, LIBLOOM refines its analysis through class-level Bloom filters. This step involves a more granular examination where each class within identified candidate packages undergoes a similar process of Bloom filter construction:

These class-level Bloom filters facilitate precise subset queries where LIBLOOM checks if the Bloom filter of an app class is a subset of the Bloom filter of a corresponding library class. Successful subset matches indicate a high probability of the library class's presence within the app, thereby confirming the integration of specific components of the TPL.

Similarity Calculation

The final step in LIBLOOM's methodology is the calculation of a similarity score based on the detailed analysis provided by the class-level subset queries. This step aggregates the results to determine the extent of library integration within the app.

The similarity score is computed as a ratio of the number of classes from the library that match classes in the app to the total number of classes in the library. This metric quantitatively reflects the degree to which a TPL is integrated into the application.

Conclusion

The primary objective of the LIBLOOM approach is to provide a scalable solution for the detection of third-party libraries in Android applications. This method, centered around the innovative use of Bloom filters for set inclusion, offers both efficiency and effectiveness, crucial for large-scale application analysis. As we move forward, we plan to test and compare LIBLOOM against other existing tools as the code is available.

3.4.7 LibScan

LibScan [23] is a tool developed by Yafei Wu et al. in 2023 to analyze Android APKs for the presence of third-party libraries, with a focus on handling code obfuscation. The tool decompiles APKs into Java classes and systematically evaluates code features, method opcodes, and call-chain opcodes to establish the extent of library integration.

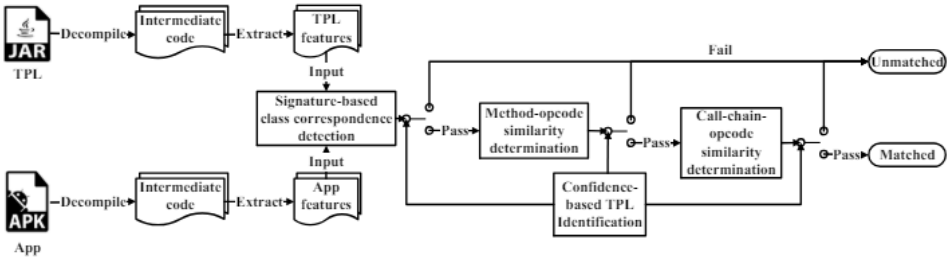


Figure 3.10: Overall architecture of LibScan.

Step 1: Decompilation

LibScan decompiles the APK file of the app and the TPL into Java classes. This step converts the compiled Android application back into Java code format, enabling detailed analysis.

Step 2: Feature Extraction

Before proceeding with specific detection steps, LibScan extracts essential information from both the app and the TPL. This includes class, field and method level information which will be used in the different analysis steps.

Step 3: Signature-based class correspondence detection

1. **Signature-Based Class Correspondence Detection**

   This initial stage involves identifying class correspondence based on invariant code features that survive obfuscation. LibScan categorizes these features into three levels:

   - **Class-Level Features:** Comprises 6 different class/interface types identified from the smali code.
   - **Field-Level Features:** Includes 22 types of smali keywords, each potentially modified by a static qualifier, totaling 44 unique features. An additional feature accounts for classes without any fields.
   - **Method-Level Features:** Defined by 736 possible combinations derived from 23 return types, two method qualifiers (static or non-static), and 24 parameter types.

   Each class in the app and TPL is assigned a signature vector consisting of Boolean variables that represent these features. Pairwise matching of these signatures determines if classes proceed to the next steps or if their correspondence is dismissed.

2. **Method-Opcode Similarity Determination**

   The second step addresses the possibility of multiple app classes corresponding to a single TPL class, which can indicate false positives.

   Each method in the app class is compared against methods in the TPL class based on their opcodes. A method from the app is matched to a TPL method if its opcode set is a subset of or identical to the TPL method's opcode set. This helps ensure that the essential functionalities of TPL methods are retained despite potential obfuscations.

   When multiple app methods correspond to a single TPL method, the best match is selected based on the minimal difference in opcode sets.

   A similarity score is computed based on the ratio of matched method sizes to the total method size in the app class. A threshold is used to decide if the class pair maintains their correspondence.

3. **Call-Chain-Opcode Similarity Determination**

   Lastly, to enhance the accuracy of method-level matches, LibScan evaluates the call chains originating from matched methods.

   For each matched method pair, LibScan performs a depth-first traversal on their respective call graphs to generate call chains, skipping standard library calls to maintain focus on relevant interactions.

   The opcodes in these call chains are then compared; a method pair is confirmed as matched only if the opcode set from the TPL method call chain is a subset of the app method's call chain.

   This step refines the detection by incorporating behavioral aspects of the methods, reducing the likelihood of false positives from mere structural

similarities.

As with other tools, the code for LibScan is readily available, enabling testing and comparison. This access allows us to better understand how effectively the tool performs, ensuring a deeper insight into its functionality and detection capabilities.

Chapter 4

Empirical comparison between tools

In this chapter, attempt to thoroughly compare the state-of-the-art TPL detection tools from the practice perspective. Due to the inherent differences in system architecture, and app distribution between iOS and Android, a cross-platform comparison of TPL detection tools is impractical. Instead, our analysis will focus on evaluating tools within the same ecosystem to establish which ones perform best in their respective environments.

Our comparative study will assess four key aspects of TPL detection tools: their effectiveness in correctly identifying TPLs, their efficiency and scalability when applied to large app datasets, their capability to pinpoint specific TPL versions, and their resilience against obfuscation techniques commonly employed in app development. These facets have been chosen to address the core attributes that define a TPL detection tool's utility in real-world scenarios.

4.1 Research experiments

With our goal of finding the best tool for iOS and Android we will be conducting this four experiments:

Experiment 1: Effectiveness Evaluation

This experiment quantifies the effectiveness of TPL detection tools using the standard metrics of precision, recall, and F1-score. These metrics are critical in evaluating the accuracy with which each tool identifies TPLs within a set of mobile applications.

- Precision is defined as the ratio of correctly identified TPLs (True Positives) to the total number of TPL identifications made by the tool.
- Recall measures the ratio of correctly identified TPLs to the total TPLs present in the ground truth data.

- F1-Score provides a harmonic mean of precision and recall, balancing the two metrics to gauge the tool's overall accuracy.

Experiment 2: Efficiency and Scalability

The second experiment assesses the efficiency of each tool by measuring the time required to detect TPLs across a standardized dataset. This measure is crucial to understand each tool's scalability and its suitability for large-scale deployments, which are common in industrial settings.

Experiment 3: Capability of Version Identification

This experiment focuses on the tools' ability to accurately identify specific versions of TPLs. Accurate version identification is vital for tasks such as vulnerability management and compliance checking. The challenges of distinguishing between closely related versions often result in false positives, which this experiment aims to quantify and analyze.

Experiment 4: Obfuscation-Resilience Capability

Given that a significant percentage of TPLs are obfuscated to protect intellectual property or prevent tampering, a tool's ability to effectively detect TPLs under these conditions is paramount. This experiment compares how each tool performs against various obfuscation strategies and identifies which tools are most adept at maintaining high detection accuracy despite these challenges.

Experiment 5: User Experience Evaluation

The final experiment will assess the user experience of the tools from a developer's perspective. Developers with varying levels of expertise will be tasked with using the tools to analyze a reduced version of the D1 dataset. They will evaluate the tools based on two aspects: how easy the tools are to use (installation, setup, and execution processes) and how easy it is to understand the results provided. Feedback will be collected through standardized questionnaires and follow-up interviews, providing insights into the user experience with each tool.

4.2 Android tools

This section focuses on evaluating five Android TPL detection tools, previously discussed in the state-of-the-art section, which have their source code available for analysis, ORLIS, despite having its source code available, we weren't able to execute it properly due to some incompatibilities, for this reason it wasn't included in the experiment.

4.2.1 Database building

For our evaluation, we created three datasets tailored to meet the requirements of the four experiments conducted, aiming to establish an empirical assessment of third-party library (TPL) detection tools.

Building a ground truth of 200 apps is time-consuming, so we leveraged existing datasets while adding 50 additional apps to ensure unbiased results. The primary dataset, D_1, was used to evaluate the effectiveness and version detection capabilities of TPL tools. This dataset includes 75 APKs from the ORLIS ground truth [24], 75 from the LIBLOOM ground truth [25], and 50 APKs manually selected from the Play Store to diversify the sample. By including a total of 529 libraries across 1,325 versions, we ensured comprehensive coverage of TPLs, providing a strong basis for our experiments.

For the efficiency and scalability assessment, we constructed Dataset D_2 by incorporating the same 50 Play Store APKs from D_1 and expanding the dataset to include 859 libraries with 5,692 distinct versions. This approach creates a more extensive environment that simulates the scale and diversity found in real-world applications, allowing us to evaluate the tools' performance under industrial-scale conditions.

In the third experiment, which focuses on version detection capabilities, we reused Dataset D_1. Its wide range of library versions allowed us to assess how well the tools could accurately identify and distinguish between versions.

To evaluate the tools resilience to obfuscation in the final experiment, we modified Dataset D_1, applying various obfuscation techniques to construct Dataset D_3. This process replicated real-world challenges by transforming the dataset into a testbed that simulates the obfuscation strategies used in practice.

App Dataset	Category	Apps	TPLs	Source
D_1	Not-Obfuscated	75	454	Orlis ground truth
		75	551	LibLoom ground truth
		50	320	Manually selected
D_2	Not-Obfuscated	50	5,692	Manually selected
D_3	ProGuard	250	1,325	Derived from D_1
	Allatori			

Table 4.1: Overview of datasets used in the experiments.

4.2.2 Effectiveness Evaluation

The evaluation of Android TPL detection tools, measured by precision and recall, revealed a range in effectiveness. Most tools exhibited high precision, indicating accurate identification of TPLs when they were detected. However, recall rates were generally low, suggesting a significant portion of TPLs remain undetected.

LIBLOOM, while exhibiting a slightly lower precision (0.6984), achieved a remarkably high recall (0.9598), underscoring its ability to capture a vast majority of TPLs. This is likely due to its two-stage approach, which first filters out unlikely candidates using package-level signatures before performing a more detailed class-level analysis. This approach allows LibLoom to efficiently process a large number of potential TPLs without sacrificing recall. The lower precision might come from the fuzzy nature of the signatures used, which could lead to some false positives.

LibScout, while not achieving the highest recall, demonstrated a good precision (0.8703) with a recall of 0.6929. This suggests that LibScout is able to identify a significant portion of the ground truth TPLs with a relatively low rate of false positives. Its reliance on a simplified Merkle tree structure and bytecode normalization could contribute to its robustness in handling code variations. However, its reliance on class hierarchy information and a fixed-depth tree structure might limit its ability to detect TPLs that have undergone significant modifications or are embedded within other libraries.

LibScan, with a balanced precision (0.7550) and recall (0.5765), showcases a reliable compromise between accuracy and robustness. Its hybrid approach, using both signature-based and similarity-based techniques, allows it to adapt to diverse TPL implementations and coding styles. The use of method opcode and call chain analysis contributes to its balanced performance, as these features are relatively stable across different library versions and implementations.

LibRadar, being a clustering-based tool, exhibited a relatively lower recall (0.4553). Its reliance on a pre-defined signature database and the nature of clustering algorithms, which might group unrelated code segments together, could explain the lower recall and the occurrence of false positives. Nevertheless, LibRadar's high precision (0.7149) signifies its accuracy in identifying TPLs that match its existing signatures. This suggests that while LibRadar might miss some TPLs, the ones it does identify are highly likely to be true positives. The reliance on stable API call frequencies as features could contribute to its precision, as these frequencies are generally resistant to modifications.

LibID, with the lowest recall among the evaluated tools (0.4782), could be attributed to its heavy reliance on package names and class hierarchies for identification. This approach might be susceptible to code modifications that remove or alter these identifiers, leading to missed detections. Additionally, while LibID employs binary integer programming to optimize the matching process, the complexity of this approach might not always yield the most accurate results, especially in cases of heavily modified code or when dealing with libraries that share similar structures.

To provide a balanced assessment of the tools' overall effectiveness, we consider the F1-score, LIBLOOM (0.8085) emerges as the top performer, followed closely by LibScout (0.7715), demonstrating a superior ability to accurately and comprehensively detect TPLs in Android applications.

Tool	Precision	Recall	F1-Score
LibRadar	0.7149	0.4553	0.5563
LibScout	0.8703	0.6929	0.7715
LibID	0.4782	0.4782	0.4782
LIBLOOM	0.6984	0.9598	0.8085
LibScan	0.7550	0.5765	0.6538

Table 4.2: Effectiveness Evaluation of Android tools

4.2.3 Efficiency Evaluation

In this section, we evaluate the efficiency of each tool, for this, a larger library dataset than the other experiments was used.

LIBLOOM is the fastest tool, with an average processing time of 1.891 seconds per app. This is due to its two-stage approach, initially filtering unlikely candidates using package-level signatures and then performing a more detailed class-level analysis only on potential matches. The use of Bloom filters for efficient set membership tests further enhances its speed. The package-level filtering step significantly reduces the number of comparisons required, resulting in a faster overall analysis.

LibScout is the second fastest tool, with an average processing time of 2.241 seconds per app. Its efficiency comes from its use of a simplified Merkle tree structure and its focus on fuzzy method signatures at the package level. This approach reduces the granularity of the analysis compared to class-level features, resulting in faster comparisons. However, LibScout also includes a class-level comparison step if an exact match isn't found at the package level, which can increase processing time for some cases.

LibRadar is the third fastest tool, with an average processing time of 8.283 seconds per app. This speed can be attributed to its unique preprocessing step, utilizing the LIBDEX tool to directly process .dex files, bypassing time-consuming reverse engineering steps. Additionally, LibRadar focuses on single DEX files, ignoring the more complex multiple DEX scenarios common in Android applications. Its reliance on pre-computed hashes of framework Android APIs further streamlines the comparison process, contributing to its efficiency.

LibScan is notably slower than the previous tools, with an average processing time of 24.395 seconds per app. This can be attributed to its deeper analysis, utilizing both method opcode and call chain information. While these features provide more accurate detection, they also require more computationally intensive comparisons. The method-opcode similarity determination necessitates pairwise comparisons between methods, while the call-chain-opcode similarity determination further increases complexity by analyzing sequences of method calls.

LibID is the slowest tool by a significant margin, with an average processing time of 1263.372 seconds per app. Its multi-phase matching process, including class matching, dependency matching, and library matching, is inherently time-consuming. The reliance on binary integer programming for optimization, while potentially improving accuracy, introduces significant computational overhead. Additionally, LibID operates at the class-level, analyzing a much larger number of code elements compared to tools that use package-level features. The fine-grained analysis performed by LibID, while potentially increasing accuracy, comes at the cost of significantly higher computational requirements.

Tool	Min Time (s)	Max Time (s)	Avg Time (s)
LibRadar	0.151	22.938	8.283
LibScout	0.255	6.469	2.241
LibID	0.912	5705.471	1263.372
LIBLOOM	0.502	20.702	1.891
LibScan	10.0	67.0	24.395

Table 4.3: Time Evaluation of TPL Detection Tools

4.2.4 Capability of Version Identification

In this section, we evaluate the effectiveness in identifying specific library versions. For this analysis, we only considered the libraries that were correctly detected in the initial effectiveness evaluation. A true positive was counted when the exact library version matched, while any detection of an incorrect library version was counted as a false positive. LibRadar wasn't included in this analysis as it can only identify libraries at the package level, making version-level identification infeasible.

LibScout exhibited a high precision (0.4968), the highest among the tools, indicating its ability to correctly identify a higher proportion of true positives out of all identified instances. Additionally, it achieved a commendable recall (0.8851), resulting in a robust F1 score of 0.6364. LibScout's performance can be attributed to its reliance on class hierarchy information and fuzzy method signatures. These techniques allow LibScout to maintain high accuracy by being less affected by minor code modifications, which are common between different versions of the same library. However, its reliance on a fixed-depth Merkle tree structure might limit its ability to capture variations in class hierarchies across different library versions, potentially leading to some missed detections and lower recall.

LibID also showed strong recall (0.9130) but a lower precision (0.4285), resulting in an F1 score of 0.5833. This suggests that LibID is very effective in identifying most TPL versions present in the dataset but also tends to generate more false positives. LibID uses binary integer programming to optimize the matching process by considering class signature containment and dependency relationships. This complex approach helps achieve high recall, but the lower precision indicates challenges in accurately distinguishing between closely related versions, possibly due to subtle changes in code structure that the algorithm might misinterpret.

LIBLOOM demonstrated a similar pattern with a high recall (0.9275) and moderate precision (0.4387), leading to an F1 score of 0.5957. LIBLOOM's ability to achieve high recall is likely due to its comprehensive code analysis approach, which includes examining method calls, class hierarchies, and control flow graphs. This thorough analysis enables LIBLOOM to detect a wide range of TPL versions effectively. However, the moderate precision suggests that the tool may over-report, identifying some false positives due to the difficulty in distinguishing between minor version changes, which often involve subtle code

modifications.

LibScan achieved the highest recall (0.9853) but the lowest precision (0.2898), resulting in an F1 score of 0.4478. This indicates that LibScan is highly effective at identifying nearly all TPL versions present but struggles with a high rate of false positives. LibScan employs a multi-step approach, including signature-based matching, method opcode similarity, and call chain opcode similarity. While these techniques are robust in capturing a wide range of TPL versions, they also lead to higher false positives because unrelated code segments may exhibit similar opcode patterns or call chains. Additionally, minor changes between library versions, such as the addition or removal of a few instructions, might not be sufficient to differentiate them using LibScan's opcode-based comparison, further contributing to false positives.

Choosing the right tool depends on the specific needs of the analysis. For comprehensive detection with some false positives, LibScan or LibLoom might be preferred. For minimizing false positives, LibScout is more suitable.

Tool	Precision	Recall	F1-Score
LibScout	0.4968	0.8851	0.6364
LibID	0.4285	0.9130	0.5833
LIBLOOM	0.4387	0.9275	0.5957
LibScan	0.2898	0.9853	0.4478

Table 4.4: Version Identification Results

4.2.5 Obfuscation-Resilience Capability

In this section, we analyze the obfuscation-resilience capability of the TPL detection tools. Obfuscation is a common technique used to protect code from reverse engineering and tampering by making it more difficult to understand. This capability is crucial for TPL detection tools, as a significant number of mobile applications use obfuscation techniques to protect their code. Evaluating how well these tools can detect TPLs in obfuscated code is essential to understanding their effectiveness in real-world scenarios.

Experimental Setup

For this experiment, we manually selected a set of 50 APKs. These APKs were then processed using two popular obfuscation tools: Allatori [26] and ProGuard [27]. Both tools are widely used in the industry for their robust obfuscation techniques.

Allatori is known for its comprehensive set of obfuscation features, which include:

- **Class and Method Renaming:** This feature changes the names of classes and methods to meaningless strings, making it difficult to understand the code flow.

- **String Encryption:** Strings within the code are encrypted, preventing easy readability of any sensitive information or logic contained in the strings.
- **Control Flow Obfuscation:** The control flow of the program is altered by adding non-linear jumps and fake branches, which complicates static analysis.

ProGuard also offers a variety of obfuscation techniques, such as:

- **Shrinking:** This removes unused classes, fields, methods, and attributes, reducing the size of the code and making it harder to reverse-engineer.
- **Optimization:** The bytecode is analyzed and optimized to improve performance and further obfuscate the code by inlining methods, removing redundant instructions, and more.
- **Renaming:** Classes, fields, and methods are renamed to short, meaningless names, which hinders understanding and analysis of the code.

Evaluation Results Analysis

The evaluation of obfuscation resilience for Android TPL detection tools, as shown in the provided F1 scores, reveals varying degrees of effectiveness in handling code obfuscation compared to their performance on unobfuscated code. The obfuscation techniques employed by Allatori and ProGuard had differing impacts on the tools' performance, highlighting the strengths and weaknesses of their respective methodologies.

Tool	Allatori			ProGuard			Non-Obfuscated
	Precision	Recall	F1 Score	Precision	Recall	F1 Score	F1 Score
LibRadar	0.5052	0.2696	0.3516	0.4807	0.2415	0.3215	0.5563
LibScout	0.5806	0.1172	0.1951	0.9411	0.2264	0.3650	0.7715
LibID	0.5625	0.3913	0.4615	0.3361	0.1689	0.2248	0.4782
LIBLOOM	0.6571	0.9019	0.7603	0.6902	0.4333	0.5324	0.8085
LibScan	0.7122	0.5857	0.6428	0.6750	0.2596	0.3750	0.638

Table 4.5: Results of Obfuscation-Resilience Capability Analysis

LIBLOOM demonstrates the highest resilience to obfuscation, with a relatively minor decrease in F1 score from 0.8085 (unobfuscated) to 0.7603 (obfuscated with Allatori) and 0.5324 (obfuscated with ProGuard). This resilience can be attributed to its two-stage approach and use of fuzzy signatures, which are robust against identifier renaming and code shrinking employed by both obfuscators. However, ProGuard's optimization techniques, which might include method inlining and dead code removal, seem to have a more significant impact on LIBLOOM's performance compared to Allatori's focus on class and method renaming. The fact that LIBLOOM does not rely on the package structure for library identification could also contribute to its resilience against obfuscation techniques that modify the package hierarchy.

LibScan experiences a more pronounced decrease in F1 score from 0.6538 (unobfuscated) to 0.6428 (obfuscated with Allatori) and 0.3750 (obfuscated with ProGuard). While its hybrid approach offers some resilience, the reliance on method opcodes and call chains might make it susceptible to Allatori's control flow obfuscation, which significantly alters the program's execution flow. ProGuard's optimization techniques, such as method inlining, could also impact LibScan's performance by changing the structure of the call chains.

LibScout suffers the most significant drop in performance, with its F1 score plummeting from 0.7715 (unobfuscated) to 0.1951 (obfuscated with Allatori) and 0.3650 (obfuscated with ProGuard). This decline suggests that LibScout's signature-based approach, which relies heavily on class hierarchy information, is highly sensitive to Allatori's class and method renaming, as well as code restructuring. ProGuard's code shrinking and optimization might have a lesser impact on LibScout compared to Allatori's more extensive structural modifications. The use of a fixed-depth Merkle tree structure in LibScout might also limit its ability to adapt to the changes introduced by obfuscation.

LibRadar also experiences a decrease in F1 score from 0.5563 (unobfuscated) to 0.3516 (obfuscated with Allatori) and 0.3215 (obfuscated with ProGuard). Allatori's obfuscation techniques, particularly string encryption and control flow transformations, likely disrupt the clustering process and hinder accurate library identification. While ProGuard's identifier renaming might also affect LibRadar, its focus on shrinking and optimization might have a less pronounced impact on the clustering algorithm. The fact that LibRadar relies on API call frequencies, which can be altered by obfuscation, could also explain its decreased performance.

LibID performance also deteriorates under obfuscation, with its F1 score dropping from 0.4782 (unobfuscated) to 0.4615 (obfuscated with Allatori) and 0.2248 (obfuscated with ProGuard). While LibID is designed to be resilient to obfuscation, Allatori's obfuscation techniques, including string encryption, control flow transformations, and method parameter changes, likely pose a greater challenge for LibID compared to ProGuard's code shrinking and optimization. The reliance on class signatures and dependency matching in LibID might be affected by Allatori's obfuscation, which could alter these relationships and hinder accurate library identification.

Overall, the results highlight the varying impacts of different obfuscation techniques on TPL detection tools. Allatori's approach, which includes string encryption, control flow transformations, and method parameter changes, seems to pose a greater challenge for most tools compared to ProGuard's more focused approach on code shrinking and optimization. This underscores the importance of considering the specific obfuscation tools and techniques used when evaluating the resilience of TPL detection tools. It also emphasizes the need for continued research and development of obfuscation-resilient techniques to ensure the effectiveness of TPL detection in real-world scenarios where obfuscation is prevalent.

4.3 iOS tools

We now shift our focus from Android to iOS and delve into the evaluation of TPL detection tools for iOS applications. Unlike the Android environment, where multiple open-source tools are available for TPL detection, the landscape for iOS is more restricted. Although several tools such as CriOS and iLibScope were discussed in the state of the art, they aren't open source, which limits their accessibility for in-depth empirical analysis.

For our study, we concentrated on LibKit, the only available open-source TPL detection tool for iOS. This section outlines the experimental setup, methodologies, and the results of our evaluation for LibKit.

4.3.1 Experimental Setup

To evaluate LibKit, we used the dataset provided by the tool's developers due to specific constraints. Analyzing iOS applications requires a jailbroken iPhone to download and decrypt the apps. Unfortunately, we were unable to meet this requirement, which made it impractical to create a new ground truth dataset. As a result, we relied on the dataset provided by LibKit for our analysis.

While this approach allowed us to conduct the evaluation, it also means that our results closely align with those reported by the LibKit developers.

4.3.2 Ground Truth

The ground truth dataset provided by LibKit is extensive and detailed, facilitating a robust evaluation of the tool's capabilities. This dataset includes:

- **Libraries:** A total of 14,043 libraries with 86,597 library versions. This comprehensive collection ensures that the evaluation covers a wide range of third-party libraries commonly used in iOS development.
- **Applications:** The dataset is divided into two subsets:
  - **Effectiveness and Time Analysis:** This subset includes 95 iOS applications. For these applications, the exact version of the libraries used isn't specified, which allows us to evaluate the general effectiveness and performance of LibKit in identifying third-party libraries within a typical set of applications.
  - **Capability of Version Identification Evaluation:** This subset comprises 43 iOS applications. For these applications, the exact versions of the libraries used are known, enabling a precise evaluation of LibKit's ability to identify the specific versions of third-party libraries.

4.3.3 Effectiveness Evaluation

The evaluation of LibKit's effectiveness in identifying specific library versions within iOS applications, as measured by precision, recall, and F1-score, reveals a nuanced performance profile. LibKit achieved a precision of 0.9183 and a

recall of 0.7887, resulting in an F1-score of 0.8487. These results are especially fine when compared to the performance of Android TPL detection tools.

LibKit's high precision can be attributed to its fingerprint-based approach, which leverages a comprehensive database of library fingerprints derived from the CocoaPods repository. By comparing the fingerprints extracted from the application's binary code with those in the database, LibKit can accurately identify known libraries with a low rate of false positives. The use of simhash for fingerprint generation further enhances precision by ensuring that similar libraries produce similar fingerprints, reducing the likelihood of misidentification. However, the reliance on class fingerprints, which capture class and method metadata but not the actual code, might lead to some false positives when dealing with library versions that have identical metadata but different code implementations.

The recall of 0.7887, while not perfect, is still a significant achievement, especially considering that LibKit is the first tool of its kind for iOS. This indicates that LibKit is able to identify a substantial portion of the TPL versions present in the applications. LibKit's ability to identify both statically and dynamically linked libraries contributes to its high recall. The tool's ability to handle partially included libraries, where only a subset of the library's code is present in the application, further enhances its recall. The main cause for the lower recall is the absence of some fingerprints in the database as well as subtle modifications in the code for some apps, added to this, the reliance on class fingerprints, which might not capture subtle differences between library, could also contribute to the lower recall.

The F1-score of 0.8487, much higher than its Android counterparts, provides a balanced assessment of LibKit's overall effectiveness. This score suggests that LibKit strikes a good balance between accuracy and completeness, making it a reliable tool for identifying TPLs in iOS applications. The high F1-score is particularly worth to note considering that LibKit is the one of the firsts tools of its kind for iOS.

Metric	**Value**
Precision	0.9183
Recall	0.7887
F1 Score	0.8487

Table 4.6: Effectiveness Evaluation Metrics for LibKit

4.3.4 Efficiency Evaluation

LibKit's average processing time of 8 minutes per app, while notably longer than the fastest Android tools, is understandable considering the scale and complexity of its library database. LibKit uses a database of 86,597 library versions, a magnitude larger than those used in Android TPL detection. This extensive database allows for comprehensive coverage but inevitably increases the computational overhead during the matching process.

Furthermore, LibKit's approach involves a thorough analysis of each app's binary code, extracting class and method features to generate fingerprints for comparison with the database. This detailed analysis, while contributing to LibKit's high accuracy, naturally requires more processing time compared to tools that employ simpler or less comprehensive techniques.

It's important to note that the time efficiency of TPL detection tools is often a trade-off with accuracy and comprehensiveness. LibKit prioritizes thoroughness and accuracy in identifying a wide range of libraries and their versions, which necessitates a more in-depth analysis and consequently longer processing times. However, the 8-minute average processing time remains reasonable for many practical scenarios, especially considering the scale of the library database and the complexity of the task.

4.3.5 Capability of Version Identification

In assessing LibKit's ability to identify not only the presence but also the specific versions of third-party libraries in iOS applications, we evaluated its performance using precision, recall, and the F1 score. LibKit achieved a precision of 0.879, a recall of 0.815, and an F1 score of 0.846. These results are notable, especially when compared to the performance of Android TPL detection tools, which often struggle to achieve high recall rates.

This high precision indicates that when LibKit identifies a library and its version, it is correct most of the time. The tool's reliance on fingerprints derived from a vast database of CocoaPods libraries contributes to this accuracy. However, the recall of 0.815 suggests that LibKit might miss identifying some library versions present in the applications. This could be due to several factors, such as the use of libraries not included in the CocoaPods repository, custom modifications made to libraries by developers, or the inherent limitations of the fingerprinting technique in distinguishing between very similar versions. As the fingerprints used by LibKit are based on class and method metadata, subtle differences between library versions, such as minor code changes or updates that don't significantly alter the metadata, might not be captured, leading to missed detections and a lower recall.

The F1 score of 0.846, a balanced metric combining precision and recall, indicates that LibKit performs well overall in library version identification. While there is room for improvement in recall, the high precision ensures that the identified versions are reliable.

Metric	Value
Precision	0.8789
Recall	0.8152
F1 Score	0.8462

Table 4.7: Version Identification Results for LibKit

4.4 User Experience Evaluation

To assess the user experience of the six TPL detection tools, a structured experiment was conducted involving five developers with varying levels of expertise. Each developer was tasked with utilizing all five tools to analyze a standardized set of Android and iOS applications. The experiment aimed to evaluate two key aspects of the tools:

- **Ease of Use:** This metric assessed how intuitive and user-friendly each tool was, considering factors such as installation, setup, and execution processes.
- **Interpretability of Results:** This metric evaluated how easily developers could understand and interpret the output generated by each tool, including the identification of libraries and their versions.

To maintain consistency and minimize bias, all developers were provided with identical instructions and, when necessary, the required datasets for the tools. This controlled environment ensured that the evaluations were based on the tools' inherent usability rather than external factors. The developers' feedback was collected through a standardized questionnaire and follow-up interviews, providing both quantitative and qualitative insights into their experiences with each tool.

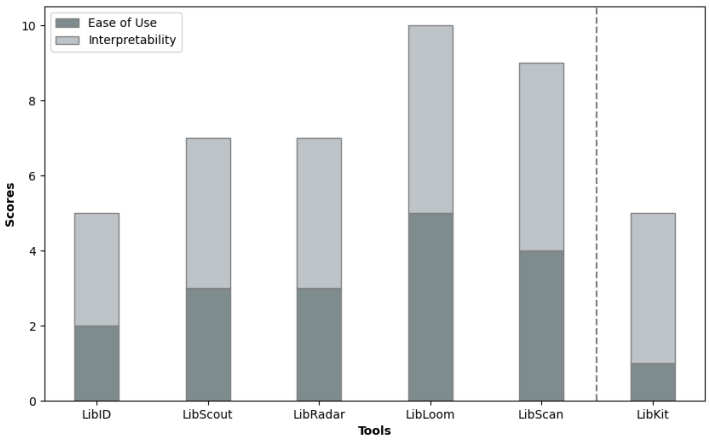


Figure 4.1: Usability Scores for each tool

Evaluation Results

Ease of Use

Overall, all participants were able to execute almost all the tools, but the complexity varied significantly between them. The primary challenges came from the tools' installation processes, particularly for those without advanced Python knowledge as five out of the six tools were written in Python, which increased

the complexity for some users.

LibID posed the most significant challenges for installation. It required the installation of the Gurobi Solver, a tool that required a license which wasn't easy to obtain. The installation process wasn't well-documented by the developers, leading to considerable difficulty for the participants. Additionally, LibID relied on older Python libraries and versions, which aren't commonly used today, further complicating the setup. However, the most critical factor affecting users ratings was the long analysis time, with some applications taking up to three hours to analyze. This duration was impractical for the participants, who couldn't afford to wait that long.

LibRadar also presented difficulties due to its reliance on a Redis server to work. Participants found setting up and configuring the Redis server an added complexity that detracted from the tool's usability.

LibScout was the only tool written in Java and required users to download the Android SDK. While participants didn't find this particularly challenging, it was noted as harder to use than the more straightforward tools.

LibScan and **LibLoom** were preferred by the users for their ease of use. These tools had more straightforward installation processes and didn't require additional complex dependencies, making them more accessible for the participants. Both tools were well-received for their simplicity and effectiveness in setup and usage.

LibKit, the iOS tool, was especially hard to use, which is reflected in it receiving the lowest score in the ease of use part. Not all users could execute the tool due to unclear instructions. Some participants mentioned encountering errors that were difficult to understand even when following the provided instructions. Additionally, users needed to dump the MongoDB to use the library database, which, despite not being inherently difficult, was poorly explained according to some users.

Ease of Understanding Results

In terms of understanding the results, all tools provided outputs that were generally clear and interpretable. However, there were differences in the verbosity and clarity of the information presented.

LibScout was noted for providing excessively verbose output. While the information was understandable, participants felt that there was too much non-essential information included, which made it harder to quickly identify the critical results.

LibID, LibRadar, LibLoom, and LibScan offered more concise and clear outputs. Among these, LibLoom and LibScan stood out for their particularly user-friendly results presentation, allowing developers to easily interpret and utilize the information provided.

LibKit achieved decent results for understanding the output. All participants who were able to execute the tool could understand the results, and they were

not excessively verbose. The participants who couldn't execute the tool were provided with the output to evaluate its understandability.

Additional Observations

- **Documentation and Support:** Participants emphasized the importance of good documentation. Tools with comprehensive and clear documentation were significantly easier to set up and use. The lack of detailed installation and usage guides was a notable drawback for some tools.
- **User Interface:** None of the tools had a graphical user interface. While this didn't significantly hinder their usage, participants suggested that even simple command-line interfaces could benefit from improved user prompts and error messages.

Chapter 5

Comparison of TPL Usage Between Android and iOS

In this chapter, we will do a comparative analysis of third-party library usage in Android and iOS applications. Using the results from the previous chapter, we selected the best tools for our analysis: **LIBLOOM** [22] for Android, due to its performance in all evaluations and ease of use, and **LibKit** [15] for iOS, as it was the only available tool for this purpose.

App Name	iOS Version	Android Version
Kickstarter	4.4.0	3.22.0
NextCloud	2.25.9	3.29.0
Wire	3.55	3.82.38
WireGuard	0.0.202001	1.0.202310
Wikipedia	6.4.1	2.7.50490
ownCloud	1.3.2	4.2.1
Tutanota	3.3.1	229.240514
Firefox	124.0	125.3.0
Seafile	2.9.6	3.0.0

Table 5.1: Apps Analysed

Third-party libraries are indispensable in modern mobile app development. They provide developers with pre-built functionalities, allowing them to focus on the unique aspects of their applications. These libraries range from UI components and network communication tools to data management solutions and security enhancements. However, the extensive use of these libraries could introduce potential risks, including security vulnerabilities, which can compromise the integrity and safety of the applications.

We begin our analysis by selecting nine apps which are in both, Android and iOS, then, using the respective tool we analyse each app obtaining the TPLs used. Understanding the extent of these dependencies provides insight into the development strategies employed across the two platforms. Typically, Android

applications incorporate a higher number of libraries due to the diverse device ecosystem and varying operating system versions they need to support. In contrast, iOS applications, operating within a more controlled environment, might exhibit a more selective use of libraries, reflecting the platform's standardized hardware and software ecosystem.

Next, we categorize the libraries based on their primary functionalities. This classification not only highlights the functional dependencies of the applications but also reveals the architectural choices and priorities of the developers. The categories include:

- **User Interface (UI) Components**: Libraries that offer pre-designed UI elements and themes, enhancing the visual appeal and usability of applications.
- **Networking**: Libraries facilitating network communications, essential for tasks such as HTTP requests, WebSocket connections, and RESTful API interactions.
- **Data Management**: Tools for managing local and cloud-based data storage, including databases, object-relational mapping (ORM) solutions, and synchronization services.
- **Image Loading and Processing**: Libraries dedicated to efficient image downloading, caching, and rendering, crucial for media-rich applications.
- **Analytics and Monitoring**: Solutions for tracking user behavior, monitoring application performance, and gathering actionable insights to drive improvements.
- **Security**: Libraries enhancing application security through encryption, secure communication protocols, and authentication mechanisms.
- **Utility Libraries**: Miscellaneous tools providing a range of utility functions, such as JSON parsing, logging, and dependency injection, simplifying common development tasks.
- **Testing Libraries**: Set of libraries that provides the tools to test the project.

Categorizing the libraries facilitates a deeper understanding of their roles in the development process and how they influence the functionality and performance of applications on each platform.

The last part of our analysis focuses on identifying how many libraries with known vulnerabilities are used by each app. Security vulnerabilities in third-party libraries pose significant risks, potentially exposing applications to exploits and attacks. By compiling a detailed list of libraries with documented vulnerabilities, including Common Vulnerabilities and Exposures identifiers and impact descriptions, we aim to raise awareness among developers and encourage proactive measures to address these issues. This analysis aims to provide insights into which platform might be more vulnerable based on the prevalence and severity of these vulnerabilities. By comparing the security records of li-

braries used in each platform, we seek to offer a clearer perspective on the relative safety of developing and maintaining applications within these ecosystems.

5.1 Number of TPL per app

Platform	Min	Max	Average
Android	6	72	35.7
iOS	1	55	17.7

Table 5.2: Number of libraries per app for Android and iOS

The analysis of third-party library usage for the same set of apps on Android and iOS reveals a significant difference in the average number of libraries used per app. On average, Android apps use approximately 35.7 libraries, while iOS apps use about 17.7 libraries. This difference highlights several key aspects of the development environments and practices on each platform.

Factors Contributing to the Difference

Fragmentation and Ecosystem Diversity

Android's ecosystem is highly fragmented, with a wide variety of devices, screen sizes, and OS versions. To ensure compatibility and a consistent user experience across all devices, developers often rely on numerous third-party libraries. These libraries help manage different hardware capabilities, optimize performance, and provide consistent UI components across various Android devices.

In contrast, iOS has a more controlled and uniform environment. Apple devices have a more standardized set of hardware and fewer variations in screen sizes and OS versions. This uniformity reduces the need for numerous third-party libraries, as the native tools and frameworks provided by Apple can handle most development needs effectively.

Native Capabilities and Development Tools

Apple's iOS development environment provides robust native tools and frameworks like CoreData. These tools are comprehensive and cover a wide range of functionalities, reducing the need for external libraries. Developers can use these native capabilities to build feature-rich applications with fewer dependencies.

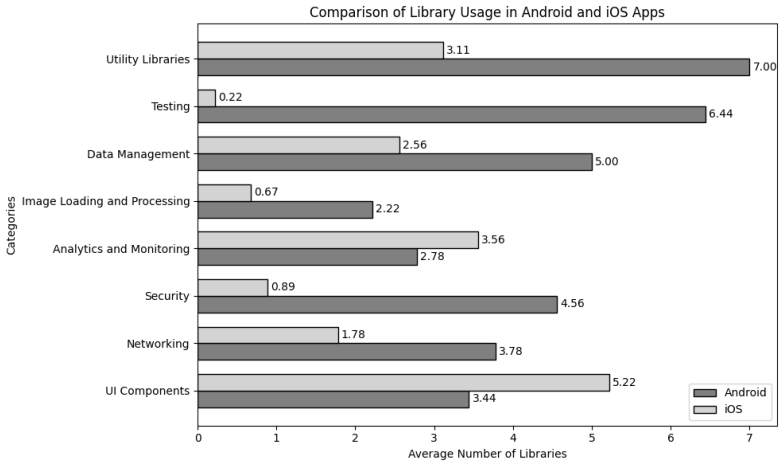
On the other hand, while Android also offers powerful native tools, the open-source nature of the platform encourages the use of third-party libraries. The Android community has developed a vast array of libraries to address specific needs, streamline development processes, and enhance app functionality. This culture of leveraging open-source resources contributes to the higher average number of libraries in Android apps.

App Review Process and Quality Control

Apple's app review process might also play a role in the lower average number of libraries in iOS apps. Developers may prefer to minimize dependencies to reduce the risk of rejection due to potential issues with third-party libraries. Fewer dependencies can lead to more straightforward maintenance and a lower likelihood of introducing security vulnerabilities.

In contrast, the Android app review process is less strict, allowing developers more flexibility in using a wide range of libraries. This flexibility can speed up development and introduce innovative features quickly, but it also increases the average number of libraries per app.

5.2 Types of TPL Utilized



The comparison of third-party library usage between Android and iOS apps across various categories reveals distinct patterns and provides insights into the development practices and ecosystem characteristics of each platform. It is important to note that this analysis only includes third-party libraries, and both Android and iOS have native libraries that were likely used as well.

User Interface Components

The data shows that iOS apps have a higher average number of libraries for UI components compared to Android apps. iOS developers often utilize a variety of UI libraries to achieve visually appealing and highly responsive user interfaces. The rich and highly customizable UI libraries available in the iOS ecosystem, such as UIKit and SwiftUI, encourage developers to incorporate additional UI components to enhance the user experience. This diversity in UI libraries helps iOS apps maintain a consistent and polished look across different devices, leveraging the standardized hardware and software environment provided by Apple. On the other hand, Android developers often rely on native UI components offered by Android's framework, which are highly versatile and customizable, potentially reducing the need for additional third-party libraries.

Networking

Android apps, on average, use more networking libraries than iOS apps. This higher usage can be attributed to the fragmented nature of the Android ecosystem, which requires developers to support a wide range of devices, network configurations, and performance capabilities. To ensure compatibility and optimize network performance, Android developers often rely on multiple networking libraries. The open-source culture of Android further promotes the development and usage of these libraries, allowing developers to build efficient and scalable networking solutions. In contrast, iOS developers tend to rely more on Apple's native networking frameworks like URLSession, which are well-integrated and provide comprehensive functionality, thereby reducing the need for third-party networking libraries.

Security

The comparison reveals that Android apps tend to use more security libraries than iOS apps. The diverse security challenges posed by the open and varied Android ecosystem necessitate the use of multiple security libraries to protect against potential vulnerabilities. Android developers often incorporate libraries for encryption, authentication, and secure data handling to safeguard their apps. In contrast, the more controlled environment of iOS, with its stringent security measures and guidelines, reduces the reliance on additional security libraries. Apple's security framework, combined with the closed nature of the iOS ecosystem, provides a secure foundation that minimizes the need for extra security measures.

Analytics and Monitoring

The average number of analytics and monitoring libraries is slightly higher in Android apps compared to iOS apps. This difference reflects the emphasis on performance monitoring and user engagement in the Android ecosystem. Android developers have access to a greater variety of analytics solutions, which are essential for gathering insights and ensuring app stability across a diverse range of devices. Libraries like Firebase Analytics and Crashlytics are commonly used to track user behavior, monitor app performance, and identify issues. The flexibility and openness of the Android platform encourage developers to leverage these tools to optimize their apps and deliver a better user experience. Meanwhile, iOS developers might lean on built-in capabilities like Apple's App Analytics and Xcode's performance monitoring tools, which offer integrated solutions for tracking app performance and user engagement.

Image Loading and Processing

Both platforms exhibit moderate usage of image loading and processing libraries, with Android showing a slightly higher average. Efficient image handling is crucial for providing a smooth user experience, especially on devices with varying performance capabilities. Android developers often use libraries like Glide and

Coil to manage image loading, caching, and processing. These libraries offer high performance and flexibility, making them popular choices for handling complex image operations. In contrast, iOS developers rely on native frameworks such as UIImage and CoreImage, which provide robust image handling capabilities, reducing the need for additional third-party libraries.

Data Management

Android apps utilize more data management libraries than iOS apps. This higher usage is due to the need for handling diverse data storage solutions and ensuring data synchronization across different Android versions and devices. The Android ecosystem offers a wide range of robust libraries for local storage, caching, and database management, enabling developers to build efficient and scalable data handling solutions. The use of multiple data management libraries helps Android apps maintain performance and reliability, even in a fragmented environment with varying hardware and software configurations. iOS developers often rely on native solutions like CoreData and UserDefaults for data management, which are well-integrated into the iOS ecosystem and provide sufficient functionality for most applications.

Testing

The average number of testing libraries is significantly higher in Android apps compared to iOS apps. This difference highlights the rigorous testing practices encouraged by the open-source culture in Android. The availability of numerous libraries for unit testing, integration testing, and UI testing allows Android developers to thoroughly test their apps across various scenarios. Tools like JUnit, Robolectric, and Mockito are main in the Android testing framework, ensuring that apps are rigorously tested for functionality, performance, and compatibility. In contrast, iOS developers tend to rely more on Apple's native testing tools and frameworks, such as XCTest, resulting in a lower average number of third-party testing libraries.

Utility Libraries

Android apps also show a higher average usage of utility libraries compared to iOS apps. The diverse needs of the Android ecosystem drive developers to incorporate a wide range of utility libraries for logging, debugging, performance optimization, and other essential tasks. These libraries provide critical tools that enhance app functionality and user experience, addressing the challenges posed by the varied hardware and software configurations in the Android environment. In contrast, iOS developers benefit from the robust set of native tools and frameworks provided by Apple, such as Xcode and Swift standard library, reducing the need for additional utility libraries.

Library	CVE	Description of Vulnerability
AFNetworking	CVE-2015-3996	This vulnerability allows an attacker to perform a man-in-the-middle (MitM) attack to intercept SSL/TLS connections due to improper validation of SSL certificates.
SQLite3	CVE-2019-5018	This vulnerability is an out-of-bounds read in the `sqlite3Select` function, which can lead to information disclosure or a denial-of-service (DoS) attack.
SSZipArchive	CVE-2023-39136	This vulnerability allows for arbitrary code execution when a specially crafted ZIP archive is opened, due to improper handling of file paths.
ZIPFoundation	CVE-2023-39138	This vulnerability allows for arbitrary code execution when a specially crafted ZIP archive is opened, due to improper handling of ZIP entries.
GCDWebServer	CVE-2019-14924	This vulnerability allows an attacker to perform a directory traversal attack, which can lead to the disclosure of sensitive information.
OpenSSL	CVE-2022-2068	This vulnerability involves a memory corruption issue in the `c_rehash` script, which can lead to arbitrary code execution or a denial-of-service (DoS) attack.

Table 5.3: Vulnerabilities in iOS Third Party Libraries

5.3 Vulnerability per Application Comparison

In our analysis, we selected six vulnerabilities (CVE) for both Android and iOS platforms. These vulnerabilities can be seen in the respective tables, 5.3 for iOS and 5.4 for Android. For this comparison, we aimed to determine how many libraries with known vulnerabilities are present in the apps we analyzed. To maintain consistency with our previous comparisons, we used the same set of apps for both Android and iOS.

It is important to note that these vulnerabilities may not be present anymore if the developers have updated the libraries to patched versions. With this analysis we want to figure out which one of the two OS is more likely to have security issues, further analysis would need to be done to check if the version of the library is affected.

The results indicated an average of 0.78 libraries which had or still have know vulnerabilities per app for iOS and 2.44 for Android. This significant difference highlights several key aspects of the development environments and practices on each platform.

One of the main reasons for this discrepancy is the difference in the number of

Library	CVE	Description of Vulnerability
OkHttp	CVE-2023-0833	This vulnerability allows for information disclosure due to improper handling of sensitive data within HTTP requests.
Gson	CVE-2022-25647	This vulnerability allows for remote code execution due to deserialization of untrusted data, leading to potential arbitrary code execution.
Commons-IO	CVE-2021-29425	This vulnerability involves a path traversal issue, allowing an attacker to access arbitrary files outside the intended directory.
Jackson-databind	CVE-2022-42004	This vulnerability allows for remote code execution due to deserialization of untrusted data, leading to potential arbitrary code execution.
Bouncy Castle	CVE-2022-45146	This vulnerability involves an issue with the implementation of cryptographic algorithms, potentially leading to information disclosure or data manipulation.
Retrofit	CVE-2018-1000850	This vulnerability involves an issue with the handling of service interfaces, potentially leading to information disclosure or improper execution of code.

Table 5.4: Vulnerabilities in Third Party Android Libraries

third-party libraries used by apps on each platform. As previously discussed, Android apps tend to use more third-party libraries on average compared to iOS apps. This higher usage increases the likelihood of including a library with a known vulnerability.

The update practices and developer awareness on each platform also play a crucial role. While both platforms encourage developers to keep their libraries up-to-date, the open-source nature of Android may lead to a higher inclusion of outdated libraries. Developers may not always update these libraries promptly, leading to a higher incidence of vulnerabilities.

Furthermore, Apple's app review process is known for its rigor and thoroughness. Developers aiming to publish their apps on the App Store must adhere to strict guidelines and undergo a detailed review process. This scrutiny helps ensure that apps are free from major security issues, including those coming from third-party libraries.

Chapter 6

Conclusion

6.1 Objectives Review

In this thesis, we set out to provide a detailed evaluation and comparison of third-party library detection tools for Android and iOS. The objectives were designed to offer both practical guidance for developers and a deeper understanding of the capabilities and limitations of these tools. Here, we review how each of these objectives was achieved:

1. **Creation of a Comprehensive Guide**: One of the primary goals was to develop a comprehensive guide that outlines the existing TPL detection tools for both Android and iOS. This objective has been successfully met. We compiled detailed information on various TPL detection tools, covering their features, and how they work. This guide serves as a valuable resource for developers, helping them make informed decisions about which tools best suit their needs.

2. **Empirical Comparison of Tools**: Conducting a series of experiments to empirically compare the performance of TPL detection tools was another key objective. We achieved this by evaluating the tools based on metrics such as precision, recall, efficiency, and resilience against code obfuscation. Through these experiments, we gathered empirical data that highlighted the strengths and weaknesses of each tool. The findings provided clear insights into the suitability of different tools for various use cases. For instance, some tools excelled in precision but lagged in efficiency, while others were robust against obfuscation but had lower recall rates. This comparative analysis is crucial for developers in selecting the most appropriate tool for their specific requirements.

3. **User Experience Evaluation**: Evaluating the usability of these tools to ensure they meet the practical needs of the development community was our final objective. We conducted a user experience study involving developers, assessing the ease of use and interpretability of the results provided by each tool. Feedback from developers offered qualitative insights into the strengths and weaknesses of each tool from a usability perspective. This

evaluation revealed that while some tools were highly effective in detection, they required significant effort to set up and interpret results. Others were user-friendly but lacked in-depth detection capabilities. This balanced assessment helps developers choose tools that not only perform well but are also easy to use.

4. **Comparison of TPL Usage Across Platforms**: Examining the differences in TPL usage between Android and iOS applications was another objective. We analyzed the same set of applications on both platforms to identify discrepancies in TPL integration and performance. The results revealed significant differences, such as the higher average number of libraries used in Android apps compared to iOS apps. These insights provide a better understanding of cross-platform development practices and highlight the impact of TPLs on the overall performance and security of mobile applications. For example, Android's fragmented ecosystem often necessitates the use of more TPLs to ensure compatibility and functionality, whereas iOS's more uniform environment allows for fewer external dependencies.

By achieving these objectives, this thesis provides a valuable resource for developers, helping them navigate the complex landscape of TPL detection in mobile application development. The insights gained from this research aid developers in selecting the most appropriate tools and contribute to ongoing improvement and innovation in the field of TPL detection.

6.2 Personal Reflections

Conducting this thesis has been a significant challenge for me, surpassing any academic work I have previously faced. When I began, my knowledge of Android and iOS development was practically nonexistent. This is precisely why I chose this topic, to gain a thorough understanding of these platforms. However, this added complexity sometimes made the process less enjoyable, especially during the initial months when the focus of the thesis was entirely different from what it ultimately became. These changes required much reflection, but now, having completed the thesis, I believe they were the best decisions.

Academically, I am very pleased to have applied a range of knowledge acquired during my master's program to this thesis, and I have learned so much more in the process. As for the final outcome, I am also very satisfied. Despite the difficulties of the initial months, I have managed to create a detailed guide for future developers. This guide helps them identify libraries in both Android and iOS applications, providing detailed explanations and an empirical comparison to help them choose what best suits their needs.

This thesis has not only been an academic exercise but also a personal journey. The initial lack of knowledge required me to immerse myself in both Android and iOS development environments, learning from the ground up. This steep learning curve was daunting, and there were moments when I doubted the feasibility of my chosen topic. However, overcoming these doubts and challenges has been

immensely rewarding. The iterative process of refining the thesis topic and focus has taught me valuable lessons in adaptability and problem-solving.

In conclusion, this project has provided me with essential knowledge for my future, and I am very pleased to have gained it. The work has been demanding, with many challenging situations that I am proud to have overcome. I am grateful for the opportunity to push my boundaries and achieve something meaningful that will benefit not only my career but also other developers in the industry.

Bibliography

[1] W. Contributors. "Ios". (2019), [Online]. Available: https://en.wikipedia.org/wiki/IOS.

[2] Apple. "Apple reports first quarter results". (2023), [Online]. Available: https://www.apple.com/newsroom/2023/02/apple-reports-first-quarter-results/.

[3] B. Dean. "Iphone users and sales stats". (2021), [Online]. Available: https://backlinko.com/iphone-users.

[4] D. Curry. "App store data". (2021), [Online]. Available: https://www.businessofapps.com/data/app-stores/.

[5] A. Inc. "Objective-c runtime". (), [Online]. Available: https://developer.apple.com/documentation/objectivec.

[6] A. Inc. "Swift.org". (), [Online]. Available: https://swift.org/about/.

[7] Apple, *Xcode - apple developer*, 2018. [Online]. Available: https://developer.apple.com/xcode/.

[8] P. Wang, Q. Bao, L. Wang, *et al.*, "Software protection on the go: A large-scale empirical study on mobile app obfuscation", in *2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)*, 2018, pp. 26–36.

[9] W. Contributors. "Android (operating system)". (2019), [Online]. Available: https://en.wikipedia.org/wiki/Android_(operating_system).

[10] D. Curry, *Android statistics*, 2021. [Online]. Available: https://www.businessofapps.com/data/android-statistics/.

[11] [Online]. Available: https://kotlinlang.org/docs/home.html.

[12] D. Wermke, N. Huaman, Y. G. Acar, B. Reaves, P. Traynor, and S. Fahl, "A large scale investigation of obfuscation use in google play", *Proceedings of the 34th Annual Computer Security Applications Conference*, 2018. [Online]. Available: https://api.semanticscholar.org/CorpusID:3413294.

[13] D. Orikogbo, M. Büchler, and M. Egele, "Crios: Toward large-scale ios application analysis", pp. 33–42, Oct. 2016. DOI: 10.1145/2994459.2994473.

[14] J. Guo, M. Zheng, Y. Zhou, *et al.*, "Ilibscope: Reliable third-party library detection for ios mobile apps", Cornell University, 2022. DOI: https://doi.org/10.48550/arxiv.2207.01837.

[15] D. Domínguez-Álvarez, A. De La Cruz, A. Gorla, and J. Caballero, "Libkit: Detecting third-party libraries in ios apps", 2023. DOI: https://doi.org/10.1145/3611643.3616344.

[16] A. Narayanan, L. Chen, and C. K. Chan, "Addetect: Automated detection of android ad libraries using semantic analysis", in *2014 IEEE Ninth International Conference on Intelligent Sensors, Sensor Networks and Information Processing (ISSNIP)*, 2014, pp. 1–6.

[17] C. Tumbleson, *Apktool*. [Online]. Available: https://apktool.org/docs/the-basics/intro/.

[18] Z. Ma, H. Wang, Y. Guo, and X. Chen, "Libradar: Fast and accurate detection of third-party libraries in android apps", in *2016 IEEE/ACM 38th International Conference on Software Engineering Companion (ICSE-C)*, 2016, pp. 653–656.

[19] M. Backes, S. Bugiel, and E. Derr, "Reliable third-party library detection in android and its security applications", in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, Vienna, Austria: Association for Computing Machinery, 2016, pp. 356–367. [Online]. Available: https://doi.org/10.1145/2976749.2978333.

[20] Y. Wang, H. Wu, H. Zhang, and A. Rountev, "Orlis: Obfuscation-resilient library detection for android", pp. 13–23, 2018.

[21] J. Zhang, A. R. Beresford, and S. A. Kollmann, "Libid: Reliable identification of obfuscated third-party android libraries", in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, Beijing, China: Association for Computing Machinery, 2019, pp. 55–65. [Online]. Available: https://doi.org/10.1145/3293882.3330563.

[22] J. Huang, B. Xue, J. Jiang, *et al.*, "Scalably detecting third-party android libraries with two-stage bloom filtering", *IEEE Transactions on Software Engineering*, pp. 2272–2284, 2023.

[23] Y. Wu, C. Sun, D. Zeng, G. Tan, S. Ma, and P. Wang, "Libscan: Towards more precise third-party library identification for android applications", in *32nd USENIX Security Symposium (USENIX Security 23)*, Anaheim, CA: USENIX Association, 2023, pp. 3385–3402. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/wu-yafei.

[24] *Orlis ground truth*. [Online]. Available: https://github.com/presto-osu/orlis-orcis/tree/master/orlis/open_source_benchmarks.

[25] *Libloom ground truth*. [Online]. Available: https://github.com/iser-mobile/LIBLOOM/tree/main/dataset.

[26] *Allatori obfuscator*. [Online]. Available: https://allatori.com/doc.html.

[27] *Proguard obfuscator*. [Online]. Available: https://github.com/Guardsquare/proguard.

[28] X. Zhan, T. Liu, L. Fan, *et al.*, "Research on third-party libraries in android apps: A taxonomy and systematic literature review", *IEEE Transactions on Software Engineering*, pp. 4181–4213, 2022.

[29] C. Soh, H. B. Kuan Tan, Y. L. Arnatovich, A. Narayanan, and L. Wang, "Libsift: Automated detection of third-party libraries in android applications", in *2016 23rd Asia-Pacific Software Engineering Conference (APSEC)*, 2016, pp. 41–48.

[30] X. Zhan, L. Fan, T. Liu, *et al.*, "Automated third-party library detection for android applications: Are we there yet?", in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, Virtual Event, Australia: Association for Computing Machinery, 2021. [Online]. Available: https://doi.org/10.1145/3324884.3416582.

[31] Y. He, X. Yang, B. Hu, and W. Wang, "Dynamic privacy leakage analysis of android third-party libraries", *Journal of Information Security and Applications*, vol. 46, pp. 259–270, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2214212618304356.

[32] *Snyk vulnerability database*. [Online]. Available: https://security.snyk.io/.